Northwestern University
Graduate Program in Department of Physics and Astronomy

PhD Prospectus

Numerical implementation and application of Gradient Ascent Pulse Engineering

by

Yunwei Lu

Abstract

Gradient Ascent Pulse Engineering (GRAPE) is a popular technique in quantum optimal control, and can be combined with automatic differentiation (AD) to facilitate on-the-fly evaluation of cost-function gradients. We illustrate that the convenience of AD comes at a significant memory cost due to the cumulative storage of a large number of states and propagators. For quantum systems of increasing Hilbert space size, this imposes a significant bottleneck. We revisit the strategy of hard-coding gradients in a scheme that fully avoids propagator storage and significantly reduces memory requirements. We benchmark runtime and memory usage and compare this approach to AD-based implementations, with a focus on pushing towards larger Hilbert space sizes. The results confirm that the AD-free approach facilitates the application of optimal control for large quantum systems which would otherwise be difficult to tackle. To facilitate the choice of optimal numerical strategy for GRAPE-based quantum optimal control, We formulate a decision tree. Based on the guidance from decision tree, we develop a GRAPE-based optimization procedure for transmon. The optimizer generates detuning-robust pulses, which are crucial in the existing superconducting quantum processor units architecture. The generated pulses outperform conventionally used transmon pulse in the aspect of infidelity robustness.

Contents

1 Introduction

The rapidly evolving field of quantum information processing has generated much interest in quantum optimal control for tasks such as state preparation [1, 2], error correction [3, 4] and realization of logical gates [5–8]. This methodology has been implemented in various quantum systems [9], such as nuclear magnetic resonance [10–15], trapped ions [16, 17], nitrogen-vacancy centers in diamonds [18–24], Bose-Einstein condensation [25–27] and neutral atoms [28, 29].

One of the prominent techniques for implementing quantum optimal control is Gradient Ascent Pulse Engineering (GRAPE). The central goal of optimal control is to adjust control parameters in such a way that the infidelity of the desired state transfer or gate operation is minimized. In GRAPE, the minimization is based on gradient descent and thus generally requires the evaluation of gradients. Automatic differentiation (AD) [30] is a convenient tool that has also made an impact on quantum optimal control with GRAPE [31–34]. Internally, AD builds a computational graph and applies the chain rule to automatically compute the gradient of a given function.

However, the convenience of AD comes at the cost of memory required for storing the full computational graph. The use of semi-automatic differentiation (semi-AD) [35] partially reduces the memory overhead compared to full automatic differentiation (full-AD). This reduction can still be insufficient to tackle optimal control for quantum systems of rapidly growing size [36, 37]. This motivates us to revisit the original GRAPE scheme [12] which is based on hard-coded gradients (HG), and has the smallest possible memory cost. We perform a scaling analysis of memory usage and runtime, and thereby develop a decision tree guiding the optimal choice of strategy among HG, semi-AD and full-AD. We exemplify the scaling behavior with benchmarks for concrete optimization tasks.

At the level of the application, Quantum optimal control helps build up quantum processor unit(QPU). In the existing superconduting QPU architecture, transmon can be used as either qubit or ancilla for bosonic qubit. In both cases, Rabi oscillation in the first two level subspace of transmon is crucial for realizing universal gate. Conventionally, to realize such oscillation in a bare transmon(without coupling to other systems), Guassian pulse is applied, where drive frequency is

chosen be on resonance with transmon frequency. In a QPU, transmon is dispersively coupled with other systems, causing stark-shifted frequency, which is state dependent. In such case, intuitively, drive frequency can be optimized to adapt such frequency shift. However, such state-dependent drive frequency can cause large hardware overhead. Another more feasible choice is to fix the drive frequency, and optimize the pulse to improve robustness of Rabis oscillation fidelity against detuning frequency. Such robust pulses are hardly obtained analytically and quantum optimal control can be implemented to find such pulsee.

Based on previous analysis of numerical impelementation of GRAPE, we develop a resource-efficient GRAPE-based optimization procedure tailored to generate detuning-robust pulses. We benchmark the optimized pulses contrast to conventially used Gaussian pulse, showing better robustness of transmon gate fidelity.

2 GRAPE based on hard-coded gradients

We briefly sketch the basics of GRAPE [12], mainly to establish the notion used in subsequent sections.

2.1 Brief sketch of GRAPE

Consider a system described by the Hamiltonian

$$H(t) = H_s + a(t)\, h_c. \tag{1}$$

Here, H_s denotes the static system Hamiltonian and h_c is an operator coupling the classical control field $a(t)$ to the system[1]. Quantum optimal control aims to adjust $a(t)$ such that a desired unitary gate or state transfer is performed within a given time interval $0 \leq t \leq T$. To facilitate optimization, the total control time T is divided into N intervals of duration $\Delta t = T/N$, and the control field is specified by the amplitudes at the discrete times $t_n = n\Delta t$ ($n = 1, 2, \ldots, N$). The discretization interval Δt is chosen such that control amplitudes are approximately constant within each Δt. This treatment is actually quite close to modeling the actual drive output of an arbitrary waveform generator, where Δt can be chosen to align with the available resolution. We denote the time-discretized values by

$$a_n \coloneqq a(t_n), \tag{2}$$

and group these to form the vector $\mathbf{a} \in \mathbb{R}^N$. Under the influence of this piecewise-constant control field, the system's quantum state $|\psi_n\rangle \coloneqq |\psi(t_n)\rangle$ evolves by a single time step according to

$$|\psi_n\rangle = U_n |\psi_{n-1}\rangle. \tag{3}$$

[1] For simplicity, we consider one control channel with real-valued $a(t)$. Generalizations to multiple control channels and complex a are straightforward.

Here, U_n is the short-time propagator

$$U_n := U(t_n, t_{n-1}) = e^{-iH_n \Delta t} \qquad (\hbar = 1), \tag{4}$$

where $H_n = H_s + a_n h_c$. Optimization of the control field proceeds via minimization of a cost function C. Typically, C includes the state-transfer or gate infidelity, along with additional cost contributions employed to moderate pulse characteristics such as maximal power or bandwidth. To this end a composite cost function is constructed, $C(\mathbf{a}) = \sum_\nu \alpha_\nu C_\nu(\mathbf{a})$, where α_ν are empirically chosen weight factors and C_ν are individual cost contributions. GRAPE utilizes the method of steepest descent to minimize $C(\mathbf{a})$ by updating the control amplitudes according to the cost function gradient:

$$\mathbf{a}^{(j+1)} = \mathbf{a}^{(j)} - \eta_j \nabla_{\mathbf{a}} C(\mathbf{a}^{(j)}). \tag{5}$$

Here, j enumerates steepest-descent iterations and $\eta_j > 0$ is the learning rate governing the step size.

2.2 Analytical gradients

A critical ingredient for GRAPE is the computation of cost function gradients. One popular option is to leverage automatic differentiation for this purpose, such as implemented in `Tensorflow` [38] and `Autograd` [39]. A clear advantage of this strategy is the ease by which complicated cost function gradients are evaluated automatically at runtime. This convenience, however, is bought at the cost of significant memory consumption which can be prohibitive for large quantum systems. Therefore, we are motivated to revisit hard-coded gradients in a computationally efficient scheme for key cost contributions.

We discuss the analytical gradients of two key cost contributions and provide detailed expressions for the gradients of other cost contributions.

In state-transfer problems, a central goal of quantum optimal control is to minimize the state-

transfer infidelity given by

$$C_1^{st} = 1 - \left|\langle\phi_T|\psi_N\rangle\right|^2, \tag{6}$$

where $|\phi_T\rangle$ is the target state and $|\psi_N\rangle$ is a state realized at final time t_N. The gradient of $C_1^{\rm st}$ takes the form

$$\frac{\partial C_1^{st}}{\partial a_n} = -\langle\phi_T|U_N\cdots\frac{\partial U_n}{\partial a_n}\cdots U_1|\psi_0\rangle\langle\psi_N|\phi_T\rangle - \text{c. c.} = -\langle\phi_n|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}\rangle\langle\psi_N|\phi_T\rangle - \text{c. c.}, \tag{7}$$

where the state $|\phi_n\rangle$ obeys the recursive relation

$$|\phi_n\rangle = U_{n+1}^\dagger|\phi_{n+1}\rangle. \tag{8}$$

This can be interpreted as backward propagation.

In addition to state transfer, unitary gates are another operation of interest. Gate optimization can be achieved by minimizing the gate infidelity $C_1^g = 1 - \left|\,\mathrm{tr}\left(U_T^\dagger U_R\right)/d\right|^2$. Here, U_T is the target unitary gate and $U_R = U_N\cdots U_1$ is the actually realized gate. For a given orthonormal basis $\{|\psi_0^h\rangle\}_h$ (h enumerates the basis states), the gradient of C_1^g can be written as

$$\frac{\partial C_1^g}{\partial a_n} = -\frac{1}{d^2}\sum_h\langle\phi_n^h|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}^h\rangle\sum_{h'}\langle\psi_N^{h'}|\phi_N^{h'}\rangle - \text{c. c.}, \tag{9}$$

where $|\psi_n^h\rangle = U_n\cdots U_1|\psi_0^h\rangle$. The state $|\phi_n^h\rangle$ is obtained as follows: apply the target unitary U_T to basis state $\{|\psi_0^h\rangle\}_h$ and backward propagate the result to time t_n,

$$|\phi_n^h\rangle = U_{n+1}^\dagger\cdots U_N^\dagger U_T|\psi_0^h\rangle. \tag{10}$$

The cost contribution

$$C_2^{st} = \frac{1}{N}\sum_{n'=1}^{N}\langle\psi_{n'}|\Omega|\psi_{n'}\rangle \tag{11}$$

penalizes the integrated expectation value of the Hermitian operator Ω. The gradient of this cost

contribution can be written as

$$\begin{aligned}\frac{\partial C_2^{st}}{\partial a_n} &= \frac{1}{N}\sum_{n'=1}^{N}\langle\psi_{n'}|\Omega\frac{\partial}{\partial a_n}\prod_{n''=1}^{n'}U_{n''}|\psi_0\rangle + c.c. \\ &= \frac{1}{N}\Big(\sum_{n'=n}^{N}\prod_{n''=n+1}^{n'}\langle\psi_{n'}|\Omega U_{n''}\frac{\partial}{\partial a_n}U_n|\psi_{n-1}\rangle\Big) + c.c. \\ &= \frac{2}{N}\,\mathrm{Re}(\langle\Phi_n|\frac{\partial}{\partial a_n}U_n|\psi_{n-1}\rangle), \end{aligned} \tag{12}$$

with

$$|\Phi_n\rangle := \sum_{n'=n}^{N} U_{n+1}^{\dagger}\cdots U_{n'}^{\dagger}\Omega|\psi_{n'}\rangle \tag{13}$$

The vector $|\Phi_n\rangle$ obeys the recursion relation

$$|\Phi_n\rangle = U_{n+1}^{\dagger}|\Phi_{n+1}\rangle + \Omega|\psi_n\rangle. \tag{14}$$

Due to this relation, the expression in Eq. (12) can be evaluated with a forward-backward scheme analogous to the one discussed in the Sec. 3.3.

The cost contribution $C_3^{st} = 1 - \frac{1}{N}\sum_{n'=1}^{N}|\langle\phi_T \mid \psi_{n'}\rangle|^2$ sums up infidelities from all time steps (as opposed to recording the final state infidelity only, as in C_1^{st}). This steers the system towards its target state more rapidly, thus helping to reduce the duration of state-transfer. Similar to Eq. (12), the gradient of C_3^{st} is:

$$\frac{\partial C_3^{st}}{\partial a_n} = -\frac{2}{N}\,\mathrm{Re}\left(\langle\Phi_{T,n}|\frac{\partial}{\partial a_n}U_n|\psi_{n-1}\rangle\right), \tag{15}$$

with

$$|\Phi_{T,n}\rangle := \sum_{n'=n}^{N} U_{n+1}^{\dagger}\cdots U_{n'}^{\dagger}|\phi_T\rangle\langle\phi_T|\psi_{n'}\rangle. \tag{16}$$

The vector obeys the recursive relation

$$|\Phi_{T,n}\rangle = U_{n+1}^{\dagger}|\Phi_{T,n+1}\rangle + |\phi_T\rangle\langle\phi_T|\psi_n\rangle. \tag{17}$$

This allows us to use a forward-backward scheme to evaluate the gradient expression in Eq. (15).

For gate operations, the cost contribution analogous to C_3^{st} is given by

$$C_3^g = 1 - \sum_{n'=1}^{N} \left| \operatorname{tr}\left(U_T^\dagger U_{n',1}\right)\right|^2 / N d^2, \tag{18}$$

where $U_{n',1} = U_{n'} \cdots U_1$ describes the evolution from time step 1 to n'. We obtain the gradient

$$\frac{\partial C_3^g}{\partial a_n} = -\frac{2}{N d^2} \sum_{h=1}^{d} \operatorname{Re}\left(\langle \Phi_{T,n}^h | \frac{\partial}{\partial a_n} U_n | \psi_{n-1}^h \rangle\right), \tag{19}$$

with

$$|\Phi_{T,n}^h\rangle := \sum_{n'=n}^{N} U_{n+1}^\dagger \cdots U_{n'}^\dagger U_T |\psi_0^h\rangle \operatorname{tr}(U_{n',1} U_T^\dagger). \tag{20}$$

Here, $\{|\psi_0^h\rangle\}$ forms an arbitrary orthonormal basis and $h = 1, 2, \ldots, d$ enumerates the basis states. The vector $|\Phi_{T,n}^h\rangle$ obeys the recursion relation

$$|\Phi_{T,n}^h\rangle = U_{n+1}^\dagger |\Phi_{T,n+1}^h\rangle + U_T |\psi_0^h\rangle \operatorname{tr}(U_{n,1} U_T^\dagger), \tag{21}$$

again enabling the evaluation of the gradient expression with a forward-backward propagation scheme.

Equations related to the gradient of cost functions such as (7) and (9) provide the analytical expressions needed for gradient descent. In the next two subsections, we will discuss how to numerically evaluate these expressions in a memory-efficient way.

3 Numerical implementation of hard-coded gradient

The calculation of gradients in equations including (7) and (9) requires numerical evaluation of expressions of the form $U_n|\Psi\rangle$ and $\frac{\partial U_n}{\partial a_n}|\Psi\rangle$. Here, the short-time propagator U_n is given by a matrix exponential e^A and $A = -iH_n \Delta t$.

Numerically evaluating $e^A|\Psi\rangle$ is not without challenge. As pointed out by Moler and Van

Loan [40], several methods exist, yet none of them is truly optimal. Three methods ranked highly in reference [40] are based on solving ordinary differential equations, matrix diagonalization and scaling and squaring (combined with series expansion). ODE solving, in this context, tends to be most expensive in runtime [35]. We mainly on focus on scaling and squaring but comment on situations where matrix diagonalization is preferable in Sec. 4.3.

3.1 State propagation evaluation by scaling and squaring method

Scaling and squaring [41] which is based on the identity

$$e^A = \left(e^{A/s}\right)^s, \tag{22}$$

where the right hand side matrix exponential now involves the matrix $B = A/s$ which has a norm that is reduced compared to $||A||$. This generally allows for a lower truncation order m of the exponential series[2] ,

$$e^B \approx e_m^B := \sum_{q=0}^{m} \frac{B^q}{q!}. \tag{23}$$

Typically, m is smaller than the one required for the original e^A. Our goal is to approximate the state vector $e^A|\Psi\rangle \approx (e_m^B)^s|\Psi\rangle$ where

$$e_m^B|\Psi\rangle = \left(1 + B + \frac{B^2}{2!} + \cdots + \frac{B^m}{m!}\right)|\Psi\rangle = |\Psi\rangle + B|\Psi\rangle + \frac{B\left(B|\Psi\rangle\right)}{2} + \cdots + \frac{B\left(\cdots\left(B|\Psi\rangle\right)\right)}{m!}\,.$$

According to the last expression, approximation of $e^A|\Psi\rangle$ can thus proceed by repeated application of $B = A/s$ to a vector, without ever invoking matrix-matrix multiplication. This boosts runtime performance from $O(d^3)$ to $O(d^2)$.

To proceed with the evaluation of $e^A|\Psi\rangle$, the truncation order m and scaling factor s are deter-

[2]While Chebyshev expansion has faster convergence than Taylor expansion, their numerical implementations have the same runtime scaling. Here, we focus on Taylor expansion due to its simplicity.

mined in such a way that the error remains below the error tolerance τ,

$$\varepsilon_{A,|\Psi\rangle}(m,s) := \frac{\left|\left|e^A|\Psi\rangle - [\![(e_m^B)^s|\Psi\rangle]\!]\right|\right|}{||e^A|\Psi\rangle||} < \tau. \tag{24}$$

Here, $[\![\bullet]\!]$ denotes the floating point representation of $\bullet$. Since this error $\varepsilon_{A,|\Psi\rangle}$ is difficult and costly to evaluate, we instead derive an upper bound $E_A(m,s)$ and resort to the inequality

$$\varepsilon_{A,|\Psi\rangle}(m,s) < E_A(m,s) \leq \tau. \tag{25}$$

3.1.1 Determine the upper bound $E_A(m,s)$

The approximation of $e^A|\Psi\rangle$ in Sec.3 required the determination of an upper bound $E_A(m,s)$ for the error $\varepsilon_{A,|\Psi\rangle}(m,s)$ [Eq. (24)]. In this appendix, we show how to acquire such bound.

Considering the expression for the error [Eq. (24)], we note that the denominator is simply 1. The error $\varepsilon_{A,|\Psi\rangle}$ is bounded as follows:

$$\begin{aligned}\varepsilon_{A,|\Psi\rangle} &= \left|\left|e^A|\Psi\rangle - (e_m^B)^s|\Psi\rangle + (e_m^B)^s|\Psi\rangle - [\![(e_m^B)^s|\Psi\rangle]\!]\right|\right|_2 \\ &\leq \underbrace{\left|\left|e^A|\Psi\rangle - (e_m^B)^s|\Psi\rangle\right|\right|_2}_{\varepsilon_t} + \underbrace{\left|\left|(e_m^B)^s|\Psi\rangle - [\![(e_m^B)^s|\Psi\rangle]\!]\right|\right|_2}_{\varepsilon_r}.\end{aligned} \tag{26}$$

The error ε_t arises from the truncation of the Taylor series and rounding error ε_r is due to finite-precision arithmetics. We proceed to derive the upper bounds for these two errors separately.

Upper bound for the truncation error ε_t We rewrite the truncation error as

$$\varepsilon_t = \left|\left|e^A|\Psi\rangle - (e^B - R_m(B))^s|\Psi\rangle\right|\right|_2 = \left|\left|\sum_{i=1}^{s} \frac{s!}{i!(s-i)!}(-R_m(B))^i(e^B)^{s-i}|\Psi\rangle\right|\right|_2, \tag{27}$$

where $R_m(B) = \sum_{q=m+1}^{\infty} B^q/q!$ denotes the matrix-valued error of the truncated exponential series. Employing the triangle inequality and submultiplicativity [42], we find the bound

$$\varepsilon_t \leq \sum_{i=1}^{s} \frac{s!}{i!(s-i)!} \big(R_m(||B||_2)\big)^i \left|\left|e^B\right|\right|_2^{s-i} |||\Psi\rangle||_2 = \sum_{i=1}^{s} \frac{s!}{i!(s-i)!} \big(R_m(||B||_2)\big)^i. \tag{28}$$

Here the second line utilizes the fact that the state is normalized and e^B is unitary.

This upper bound must be evaluated numerically to determine m and s. However, computing the matrix 2-norm is inconvenient since it requires the use of an eigensolver. To mitigate this, we switch to the matrix 1-norm by exploiting[3]

$$||B||_2 \leq ||B||_1 . \tag{29}$$

Monotonicity of R_m leads us to

$$\varepsilon_t \leq \sum_{i=1}^{s} \frac{s!}{i!(s-i)!} \big(R_m(||B||_1)\big)^i < sR_m(||B||_1) \frac{1 - \big(sR_m(||B||_1)\big)^s}{1 - sR_m(||B||_1)}. \tag{30}$$

Upper bound for the rounding error ε_r The rounding error ε_r originates from the finite precision of the floating-point representation of real numbers as well as the finite accuracy of basic arithmetic operations. The floating-point representation $[\![\xi]\!]$ for the real number ξ can be written as $[\![\xi]\!] = \xi(1+\delta)$ where $\delta = \delta(\xi)$ is the relative deviation [42]. Its magnitude is always smaller than machine precision, i.e. $|\delta| < u$. Similarly, the complex number z obeys

$$[\![z]\!] = z(1+\delta) \tag{31}$$

[3]Generally, $||B||_2 \leq \sqrt{||B||_\infty ||B||_1}$ [43] holds true for any B. When B is anti-Hermitian, $||B||_\infty = ||B||_1$ leads to inequality Eq. (29).

with $|\delta| \leq u$. Arithmetic operations such as addition and multiplication with two complex numbers z_1, z_2 incur a relative deviation $\delta = \delta(z_1, z_2)$ such that

$$[\![z_1 + z_2]\!] = ([\![z_1]\!] + [\![z_2]\!])(1+\delta),\ |\delta| \leq u, \quad [\![z_1 z_2]\!] = [\![z_1]\!][\![z_2]\!](1+\delta),\ |\delta| \leq u', \tag{32}$$

where $u' := 2\sqrt{2}u/(1-2u)$ [42]. The relative deviation shown in Eqs. (31), (32) generally depend on both numbers involved as well as the operation in question. In the following, we will distinguish different δ with subscript ν.

Using the Eqs. (31), (32), we derive an upper bound for the rounding error in the evaluation of the dot product which is the basic arithmetic operation in evaluating $e_m^B|\Psi\rangle$ according to Eq. (24).

Upper bound for the rounding error associated with evaluating the dot product The rounding error incurred when evaluating the dot product is given by $|\Sigma_d - [\![\Sigma_d]\!]|$ with $\Sigma_d = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{z}$ ($\boldsymbol{w}, \boldsymbol{z} \in \mathbb{C}^d$). Here, $\boldsymbol{w}^{\mathrm{T}}$ is a row of $B = i\delta t H/s$. We illustrate how to bound the rounding error for $d = 1, 2$, and give the general result subsequently. Based on Eqs. (31) and (32), we have

$$|[\![\Sigma_1]\!] - \Sigma_1| = |[\![w_1 z_1]\!] - w_1 z_1| < |w_1||z_1|\Big|\prod_{\nu=1}^{3}(1+\delta_\nu) - 1\Big| < \gamma_3 |w_1||z_1|. \tag{33}$$

In the last step, we have used $\left|\prod_{\nu=1}^{n}(1+\delta_\nu) - 1\right| < \frac{nu'}{1-nu'}$ [42], and defined $\gamma_n := \frac{nu'}{1-nu'}$. For $d = 2$, the upper bound of the error is

$$|[\![\Sigma_2]\!] - \Sigma_2| = |([\![\Sigma_1]\!] + [\![w_2 z_2]\!])(1+\delta_1) - \Sigma_2| < \gamma_4 \sum_{i=1}^{2} |w_i||z_i|. \tag{34}$$

The general upper bound for arbitrary d is given by:

$$|[\![\Sigma_d]\!] - \Sigma_d| < \gamma_{d+2} \sum_{j=1}^{d} |w_j||z_j|. \tag{35}$$

Since $\boldsymbol{w}^{\mathrm{T}}$ is proportional to a row of the Hamiltonian matrix, sparsity of H implies sparsity of $\boldsymbol{w}^{\mathrm{T}}$. Vanishing entries in $\boldsymbol{w}^{\mathrm{T}}$ is special because they do not incur rounding errors. Thus, one can obtain a stronger upper bound

$$|[\![\Sigma_d]\!] - \Sigma_d| < \gamma_{\sigma+2} \sum_{j=1}^{d} |w_j||z_j| \tag{36}$$

where σ is the number of non-zero elements in $\boldsymbol{w}^{\mathrm{T}}$.

Upper bound for the rounding error of $e_m^B|\Psi\rangle$ The vector $e_m^B|\Psi\rangle$ is evaluated recursively via

$$b_j = \frac{B}{j} b_{j-1}, \qquad e_j^B b_0 = b_{j-1} + b_j \tag{37}$$

where $j = 1, 2, \cdots, m$ and b_0 is a vector representation of $|\Psi\rangle$ under a choice of orthonormal basis. Before evaluating the upper bound for the error $\left|\left|[\![e_m^B b_0]\!] - e_m^B b_0\right|\right|_2$, we first derive an upper bound for the error of an vector component $\left|[\![e_m^B b_0]\!]^{(i)} - (e_m^B b_0)^{(i)}\right|$. We illustrate how to bound this error for $m = 0, 1$, and then give the general result.

For the case $m = 0$, we have the bound

$$\left|[\![e_0^B b_0]\!]^{(i)} - (e_0^B b_0)^{(i)}\right| = \left|[\![b_0]\!]^{(i)} - b_0^{(i)}\right| < \gamma_2 |b_0^{(i)}|. \tag{38}$$

We then obtain the upper bound of the error for the example of $m = 1$:

$$\begin{aligned} |[\![e_1^B b_0]\!]^{(i)} - (e_1^B b_0)^{(i)}| &= \left|\left([\![b_0]\!]^{(i)} + [\![b_1]\!]^{(i)}\right)(1+\delta_1) - (e_1^B b_0)^{(i)}\right| \\ &= \left|[\![b_0]\!]^{(i)}(1+\delta_1) + \frac{[\![Ab_0]\!]^{(i)}}{s} \prod_{\nu=1}^{3}(1+\delta_\nu)^3 - (e_1^B b_0)^{(i)}\right| \overset{(36)}{<} \gamma_3 |b_0^{(i)}| + \gamma_{\sigma_i+5} \sum_l |B^{(il)}||b_0^{(l)}|, \end{aligned} \tag{39}$$

where σ_i is number of non-zero elements in ith row of the matrix B. Here, the third line arises from the division error between a complex number and a real number, i.e. $[\![z/\xi]\!] = z/\xi(1+\delta)$,

$|\delta| < u$ [42]. The general upper bound for arbitrary m is given by:

$$\left|[\![e_m^B b_0]\!]^{(i)} - (e_m^B b_0)^{(i)}\right| < \sum_{k=0}^{m} \gamma_{k(\sigma_i+2)+m+2} \left(\frac{|B|^k}{k!}|b_0|\right)^i, \tag{40}$$

where $|B|, |b_0|$ denote the matrix and the vector with elements $|B^{il}|$, $|b_0^i|$, respectively. Defining

$$\sigma' := \max_i \sigma_i \tag{41}$$

and using Eq. (40), we obtain

$$\begin{aligned} \left|\left|[\![e_m^B b_0]\!] - (e_m^B b_0)\right|\right|_2 &= \left|\left||[\![e_m^B b_0]\!] - (e_m^B b_0)|\right|\right|_2 < \sum_{k=0}^{m} \gamma_{k(\sigma'+2)+m+2} \left|\left|\frac{|B|^k}{k!}|b_0|\right|\right|_2 \\ &< \sum_{k=0}^{m} \gamma_{k(\sigma'+2)+m+2} \frac{\left|\left||B|\right|\right|_2^k}{k!} \left|\left||b_0|\right|\right|_2 \overset{(29)}{<} \sum_{k=0}^{m} \gamma_{k(\sigma'+2)+m+2} \frac{\left|\left||B|\right|\right|_1^k}{k!} \left|\left||b_0|\right|\right|_2 = \beta ||b_0||_2 \end{aligned} \tag{42}$$

with $\beta := \sum_{k=0}^{m} \gamma_{k(\sigma'+2)+m+2} \frac{||B||_1^k}{k!}$. Here, $\beta||b_0||_2$ the desired upper bound for the rounding error of $e_m^B|\Psi\rangle$.

Upper bound for the error ε_r The vector $(e_m^B)^s b_0$ is evaluated by the recursive relation.

$$c_l = e_m^B c_{l-1}, \qquad l = 1, 2, \cdots, s, \tag{43}$$

where $c_0 = b_0$. We illustrate how to bound the error $\varepsilon_r = \left|\left|[\![(e_m^B)^s b_0]\!] - (e_m^B)^s b_0\right|\right|_2$ for $s = 1, 2$, and give the general result subsequently.

For $s = 1$, we have

$$\left|\left|[\![e_m^B b_0]\!] - e_m^B b_0\right|\right|_2 \overset{(42)}{<} \beta ||b_0||_2 = \beta. \tag{44}$$

For $s = 2$, we obtain

$$\begin{aligned}\left|\left|[\![(e_m^B)^2 b_0]\!] - (e_m^B)^2 b_0\right|\right|_2 &\leq \left|\left|[\![(e_m^B)^2 b_0]\!] - e_m^B [\![e_m^B b_0]\!]\right|\right|_2 + \left|\left|e_m^B [\![e_m^B b_0]\!] - (e_m^B)^2 b_0\right|\right|_2 \\ &< \left|\left|[\![e_m^B [\![e_m^B b_0]\!]]\!] - e_m^B [\![e_m^B b_0]\!]\right|\right|_2 + \beta \left|\left|e_m^B\right|\right|_2, \end{aligned} \tag{45}$$

where the last step is enabled by Eq. (44). Finally, we bound the matrix norm in the second term by

$$\left|\left|e_m^B\right|\right|_2 = \left|\left|e^B - R_m(B)\right|\right|_2 \leq 1 + R_m(||B||_2) \overset{(29)}{\leq} 1 + R_m(||B||_1) =: \alpha. \tag{46}$$

Since Eq. (42) holds for any complex vector b_0, the error in Eq. (45) is further bounded by:

$$\begin{aligned}\left|\left|[\![(e_m^B)^2 b_0]\!] - (e_m^B)^2 b_0\right|\right|_2 &\overset{(42)}{<} \beta \left|\left|[\![e_m^B b_0]\!]\right|\right|_2 + \beta\alpha = \beta \left|\left|[\![e_m^B b_0]\!] - e_m^B b_0 + e_m^B b_0\right|\right|_2 + \beta\alpha \qquad (47) \\ &\overset{(44)}{<} \beta(\beta + \alpha) + \beta\alpha. \qquad (48)\end{aligned}$$

Generalizing this strategy further, we obtain the expression for the upper bound of ε_r:

$$\begin{aligned}\varepsilon_r = \left|\left|[\![(e_m^B)^s b_0]\!] - (e_m^B)^s b_0\right|\right|_2 &\leq \sum_{l=0}^{s-1} \left|\left|(e_m^B)^l [\![(e_m^B)^{s-l} b_0]\!] - (e_m^B)^{l+1} [\![(e_m^B)^{s-l-1} b_0]\!]\right|\right|_2 \\ &< \beta \sum_{l=0}^{s-1} (\alpha + \beta)^{s-l-1} \alpha^l = (\alpha + \beta)^s - \alpha^s. \end{aligned} \tag{49}$$

Combining the upper bounds for ε_r and ε_t, we return to Eq. (26) and obtain

$$\varepsilon_{A,|\Psi\rangle} < E_A(m, s) = s R_m(||B||_1) \frac{1 - \left(s R_m(||B||_1)\right)^s}{1 - s R_m(||B||_1)} + (\alpha + \beta)^s - \alpha^s,$$

where $||B||_1 = ||A||_1 / s$, $\alpha = \alpha(||A||_1, m, s)$ and $\beta = \beta(\sigma', ||A||_1, m, s)$.

3.1.2 Choosing scaling factor and truncation order

For a given A when computing state propagation by scaling and squaring discussed in Sec. 3.1, inequality 25 generally has multiple solutions for m and s. As shown in 3.1.1, the evaluation

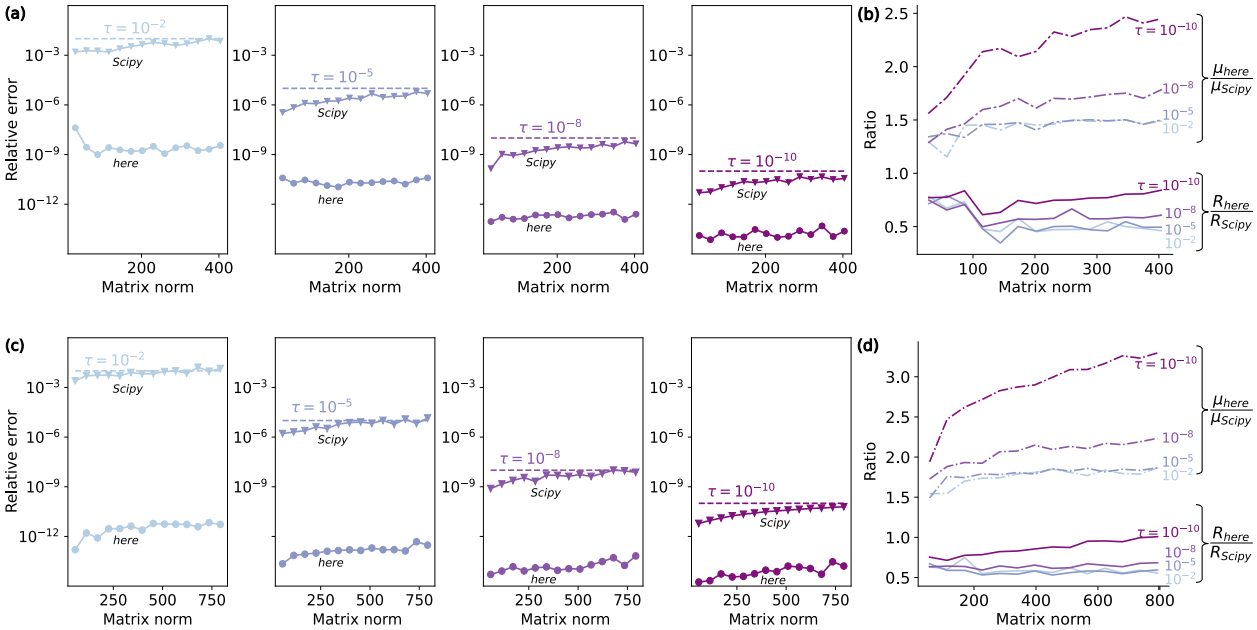

Figure 1: Comparison between scaling and squaring as implemented here and `Scipy`'s `expm_multiply`, considering relative error and runtime. (a), (c) Relative error versus matrix norm. (b), (d) Ratios of runtime and μ (i.e. number of matrix-vector multiplications) between our and `Scipy`'s implementations as functions of the matrix norm. The matrix norm $||-iH\Delta t||_2$ is varied by increasing $\Delta t \in [1, 10]$. In all panels, colors encode different error tolerances τ. The results of the first and second row are obtained based on cavity-transmon and three coupled transmons Hamiltonians, respectively. The system parameters of these two Hamiltonian are the same as the ones given in the captions of Figs. 3, 4, respectively.

requires ms matrix-vector multiplications, which are observed to dominate the runtime. Since systematic minimization of ms requires significant runtime itself, we instead: (1) select solutions to Eq. (25) with the smallest s which is denoted by $\mathbb{s}$, and (2) among these pairs, choose the one with smallest m (which we denote by $\mathbb{m}$).

Thus, calculating $e^A|\Psi\rangle$ requires

$$\mu := \mathbb{m}\mathbb{s} \tag{50}$$

matrix-vector multiplications. As shown in the following, for two concrete examples, our method for evaluating $e^A|\Psi\rangle$ can achieve better runtime performance and accuracy than `Scipy`'s `expm_multiply` function [44] .

3.1.3 Numerical comparison between scaling and squaring implementations

Scaling and squaring is used to evaluate the propagator-state product, $e^{-iH\Delta t}|\Psi\rangle$. We compare relative error and runtime between our implementation of scaling and squaring and `Scipy`'s implementation, `expm_multiply` [44]. The relative errors for our implementation and `Scipy`'s implementation follow the general expression in Eq. (24), but differ in specific choice of m and s. As a proxy for the exact propagator-state product, we apply Taylor expansion to matrix exponential with 400 digits precision by `Sympy` and the result of the product converges at 30 digits after decimal point. The runtime is measured with the help of the `Temci` package [45].

As the basis for this comparison, we use the examples of Hamiltonians (54) and (55) and implement sparse storage for them. The relative errors are shown in Fig. 1(a) and (c) as a function versus the matrix norm $||-iH\Delta t||_2$ respectively. In both examples, the relative errors for our implementation are bounded by the error tolerance τ, while the relative errors for `Scipy`'s implementation tend to exceed the tolerance for large norms. There are two reasons for this tendency. First, rounding errors are not considered in the derivation of the upper bound for the errors [44]; second the truncation of e_m^B [Eq. (23)] is merely based on a heuristic criterion.

Even though the relative errors of `Scipy`'s implementation are worse, it gains smaller number of matrix-vector multiplications, which is denoted by μ. This is illustrated in Fig. 1(b) and (d), that the ratio of μ between our implementation and `Scipy`'s implementation is always larger than 1. Although `Scipy`'s implementation spends less time on matrix-vector multiplications, the total runtime of `Scipy`'s implementation is larger than the one of our implementation, see Fig. 1(b) and (d). This is due to the larger runtime overhead of `Scipy`'s implementation for determining (m, s).

In conclusion, our implementation has the better numerical stability and runtime than `Scipy`'s implementation in our numerical experiments.

3.2 Computing propagator derivative acting on a state: auxiliary matrix method

Having addressed the challenges associated with numerically evaluating $e^A|\Psi\rangle$, we now turn to the calculation of $\frac{\partial U_n}{\partial a_n}|\Psi\rangle$, which is crucial for computing cost function gradients as in Eqs. (7) and (9). We base this evaluation on the approximation

$$\frac{\partial U_n}{\partial a_n}|\Psi\rangle \approx \frac{\partial (e_m^{B_n})^s}{\partial a_n}|\Psi\rangle, \tag{51}$$

where $B_n = -iH_n\Delta t/s$. We proceed by using the auxiliary matrix method based on the identity [46, 47]

$$(e_m^{\mathcal{A}_n/s})^s \begin{pmatrix} 0 \\ |\Psi\rangle \end{pmatrix} = \begin{pmatrix} \frac{\partial (e_m^{B_n})^s}{\partial a_n}|\Psi\rangle \\ (e_m^{B_n})^s|\Psi\rangle \end{pmatrix}. \tag{52}$$

Here, $\mathcal{A}_n$ is defined as

$$\mathcal{A}_n := \begin{pmatrix} -iH_n\Delta t & -ih_c\Delta t \\ 0 & -iH_n\Delta t \end{pmatrix}, \tag{53}$$

which is a 2×2 matrix of operators each acting on our Hilbert space with dimension d. Once a basis is chosen, $\mathcal{A}_n$ can be represented as a $2d \times 2d$ matrix of complex numbers. Numerical evaluation of the left hand side of the identity (52) gives a $2d$ dimensional vector. The first d entries of the vector correspond to $\partial (e_m^{B_n})^s/\partial a_n|\Psi\rangle$, which is an approximation of $\frac{\partial U_n}{\partial a_n}|\Psi\rangle$.

3.3 Efficient evaluation of hard-coded gradients

The analytical gradient expressions, such as ∇C_1^{st} in Eq. (7), are generally complicated and involve a large number of contributing quantities. Depending on grouping and ordering of operations, these quantities may need to be stored simultaneously or computed repeatedly with critical implications for runtime and memory usage. The scheme originally proposed by Khaneja *et al.* [12] strikes a favorable compromise that avoids the proliferation of stored intermediate state vectors. More specifically, the final quantum state $|\psi_N\rangle$ is obtained by forward propagating $|\psi_0\rangle$. Then, backward

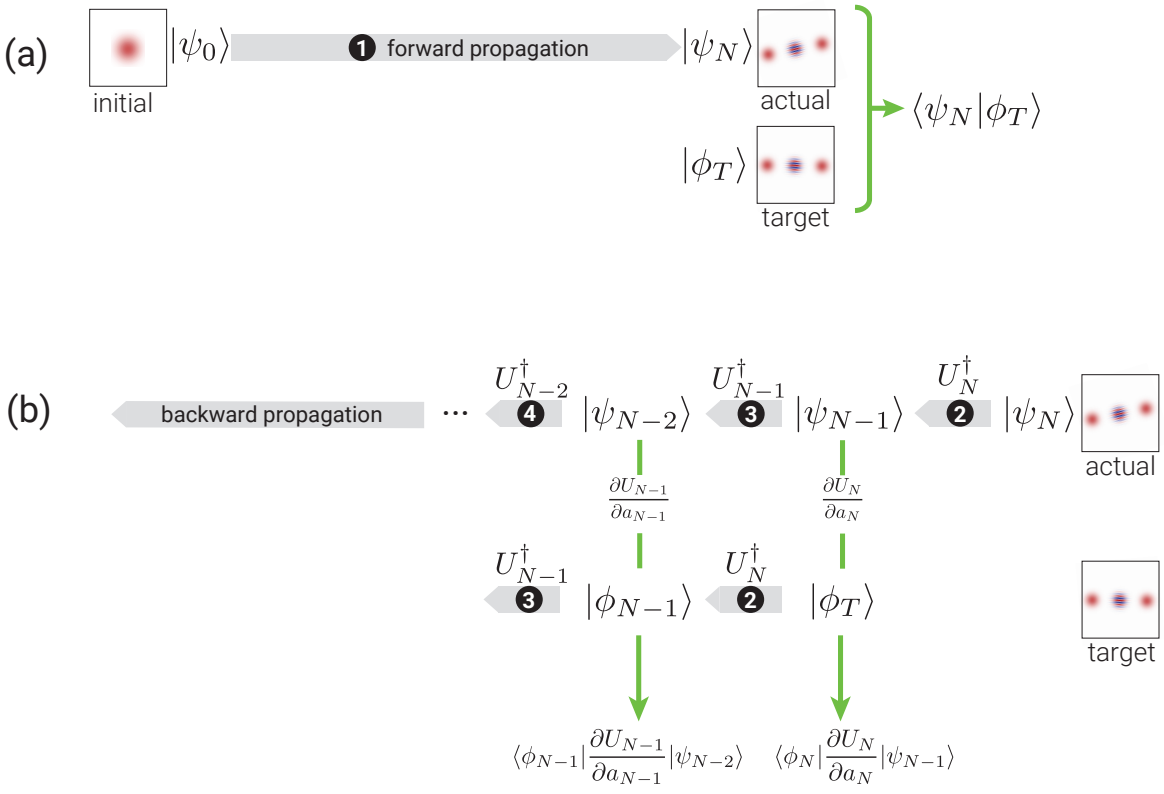

Figure 2: The forward-backward propagation scheme to compute the components of the gradient in Eq. (7). (a) Forward propagate the initial state by $U_N \cdots U_1$ to obtain the realized state $|\psi_N\rangle$ and calculate the overlap $\langle\psi_N|\phi_T\rangle$, where $|\phi_T\rangle$ is the target state. (b) Backward propagate $|\psi_N\rangle$ and $|\phi_T\rangle$ by the propagators $U_N^\dagger, U_{N-1}^\dagger, \cdots, U_1^\dagger$ iteratively, and compute $\langle\phi_n|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}\rangle$ at each step step.

propagation of $|\psi_N\rangle, |\phi_T\rangle$ is carried out to calculate the matrix element $\langle\phi_n|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}\rangle$ needed for the n-th component of the gradient, thus building up the gradient in descending order. For both propagation directions, only the current intermediate states are stored, leading to a memory cost of $\Theta(1)$.[4]

[4]Here, $f(x) = \Theta(g(x))$ is the usual Bachmann–Landau notation for $f(x)$ bounded above and below by $g(x)$ in the asymptotic sense.

4 Scaling analysis and benchmarking

We consider the following three methods to compute gradients within GRAPE: evaluation of hard-coded gradients, full automatic differentiation [31], semi-automatic differentiation [35]. To determine the method most appropriate for a given problem, we inspect the memory usage and runtime scaling of these three approaches. Following this analysis, we present benchmark studies of specific state-transfer and gate-operation problems to illustrate the scaling behavior.

4.1 Scaling analysis of runtime and memory usage

Runtime and memory usage can pose significant challenges when performing optimization tasks for quantum systems with large Hilbert space dimension (d) or for time evolution requiring significant number of time steps (N). In this subsection, we analyze the scaling of memory usage and runtime with respect to both N and d, focusing on the cost contribution C_1 for state-transfer and unitary-gate optimization. Our findings can be generalized to other cost contributions beyond C_1.

Evaluation of hard-coded gradients (HG) using forward-backward propagation. We start by considering state-transfer problems. The *memory usage* for evaluating ∇C_1^{st} is mainly determined by the required storage of quantum states and the Hamiltonian. In forward-backward propagation, the number of states and instances of Hamiltonian matrices stored at any given time is of the order of unity. Each individual state vector of dimension d consumes memory $\sim \Theta(d)$. Memory usage for the Hamiltonian depends on the employed storage scheme. In the "worst" case of dense matrix storage, the memory required for Hamiltonian is $\Theta(d^2)$. For sparse matrix storage, memory usage generally depends on sparsity and is determined by the number of stored Hamiltonian matrix elements (here denoted as κ). For example, for a diagonal or tridiagonal matrix, memory usage is $\kappa \sim \Theta(d)$; for a linear chain of qubits with nearest-neighbor σ_z coupling, it is $\Theta(d \log_2 d)$. Combining the scaling relations above gives us an overall asymptotic behavior of memory usage $\Theta(d + \kappa)$.

We observe that *runtime* of the scheme is dominated by the required $\Theta(N)$ evaluations of

propagator-state products and their derivative counterparts. Numerically approximating a single propagator-state product requires μ matrix-vector multiplications [see Eq. (50)], where runtime of one instance of matrix-vector multiplication scales as $\Theta(\kappa)$. The implicit dependence of $\mu = \mu(d)$ on d is specific to each particular system and can be difficult to obtain analytically. Numerically, we find for a driven transmon coupled to a cavity that $\mu \sim \Theta(\sqrt{d})$; for a linear chain of coupled qubits, the scaling is $\Theta(\log_2 d)$, see A for more details. Once combined, the above individual scaling relations lead to an overall runtime asymptotic behavior $\Theta(N\mu\,\kappa)$. For example, in the case of the linear chain of coupled qubits, runtime scales as $\Theta(Nd\,(\log_2 d)^2)$.

We next turn to gate-optimization problems, where scaling behavior of memory usage and runtime are as follows. We calculate ∇C_1^g by sequentially applying forward-backward propagation to each of the d basis states. The memory usage is $\Theta(d + \kappa)$, the same as for ∇C_1^{st}. The runtime scaling is $\Theta(N\mu\kappa d)$ which is d times larger than the runtime scaling for ∇C_1^{st}. (Another way of calculating ∇C_1^g consists of parallelized forward-backward propagation of all basis states. The concrete scaling behavior then depends on the parallelization scheme; that discussion is beyond the scope of this paper.)

Thanks to the similarities in the expressions and the existence of recursion relations, forward-backward schemes akin to Fig. 2 can also be applied to ∇C_2^{st},∇C_3^{st} and ∇C_3^g. As a result, we find that the scaling analysis in Sec. 4.1 leads to the same the memory usage and runtime requirement as ∇C_1^{st} and ∇C_2^g.

Full automatic differentiation (full-AD) Reverse-mode automatic differentiation extracts gradients by a forward pass and a reverse pass through the computational tree. With the forward pass the cost function C is computed and the backward pass accumulates the gradient via chain rule.

The *memory usage* for evaluating ∇C_1^{st} is determined by the requirement that all intermediate numerical results must be stored as part of the forward pass and act as input to the reverse pass. Specifically, as shown in Fig. 2, the forward pass evolves the initial state vector to the final state vector in N time steps. The total number of vectors that have to be stored along the way is $\Theta(N\mu)$,

since computation of a single short-time propagator-state product via scaling and squaring necessitates μ iterations. In addition, considering the memory usage for the Hamiltonian, the full scaling is $\Theta(N\mu d + \kappa)$.

The *runtime* for evaluating ∇C_1^{st} combines forward and reverse pass. It is dominated by the evaluation of $\Theta(N)$ propagator-state products, each of which has the runtime scaling $\Theta(\mu\kappa)$, leading to an overall runtime scaling of $\Theta(N\mu\kappa)$.

For gate-optimization problems, C_1^g is evaluated by evolving not only a single initial state but a whole set of d basis vectors. Evaluating Nd propagator-state products requires $\Theta(N\mu d^2 + \kappa)$ memory, where the number of stored Hamiltonian matrix elements κ takes into account storage of the Hamiltonian. Since κ has the worst-case scaling $\Theta(d^2)$ in a case of dense Hamiltonian matrix, the overall scaling is $\Theta(N\mu d^2)$. Assuming that the evolution of individual basis vectors is not parallelized, runtime also grows by a factor of d, so that runtime scaling for ∇C_1^g is $\Theta(N\mu\kappa d)$.

Despite the specific differences among the expressions of cost contributions C_2^{st}, C_3^{st} and C_3^g, we find that the scaling of runtime and memory usage is again set by the need to store intermediate vectors and repeatedly perform matrix-vector multiplications. Consequently, the scaling remains the same as for the cost contributions discussed in Sec. 4.1

Semi-automatic differentiation (semi-AD) For gradients of cost contributions that are tedious to derive analytically, semi-automatic differentiation [35] offers an alternative approach in which some derivatives are hard-coded and others treated by automatic differentiation. Compared to full-AD, semi-AD has the benefit that the memory usage scaling is independent of μ while maintaining the same runtime scaling.

Comparison The results of this scaling analysis are summarized in Tab. 1. For full-AD, memory usage grows linearly with respect to μ and number of times steps N. By contrast, memory usage for HG does not increase with μ and N, but remains constant. This advantage is not bought at the cost of worse runtime scaling. Therefore, HG is preferable over full-AD in the optimal control of large quantum systems where memory usage would otherwise be prohibitively large. Semi-AD

Table 1: Memory usage and runtime scaling with respect to Hilbert space dimension d, number of time steps N, the number of stored Hamiltonian matrix elements $\kappa = \kappa(d)$ and the number of matrix-vector multiplications required for evaluating propagator-state products $\mu = \mu(d)$. (The scaling of μ and κ depends on the specific optimization task.) The table contrasts scaling for state transfer and gate operation for hard-coded gradients (HG), full automatic differentiation (full-AD) and semi-automatic differentiation (semi-AD).

	STATE TRANSFER		**GATE OPERATION**	
	Runtime	**Memory usage**	**Runtime**	**Memory usage**
HG	$\Theta(N\mu\kappa)$	$\Theta(d+\kappa)$	$\Theta(N\mu\kappa d)$	$\Theta(d+\kappa)$
Full-AD	$\Theta(N\mu\kappa)$	$\Theta(N\mu d+\kappa)$	$\Theta(N\mu\kappa d)$	$\Theta(N\mu d^2)$
Semi-AD	$\Theta(N\mu\kappa)$	$\Theta(Nd+\kappa)$	$\Theta(N\mu\kappa d)$	$\Theta(Nd^2)$

presents an interesting compromise viable whenever the memory bottleneck is not too severe. We note that the runtime scaling $\Theta(N\mu\kappa d)$ of HG via scaling and squaring can in principle exceed the scaling $O(Nd^3)$ for matrix exponentiation based on matrix diagonalization (see B). In this case, the latter is the better option.

We illustrate the discussed scaling by benchmark studies for concrete optimization problems in the following section.

4.2 Benchmark studies

Our first benchmark example studies the following state-transfer problem. Consider a setup in which a driven flux-tunable transmon is capacitively coupled to a superconducting cavity, and perform a state transfer from the cavity vacuum state to the 20-photon Fock state. The Hamiltonian in the frame co-rotating with the transmon drive is

$$H_1(t) = \Delta b^\dagger b + \frac{1}{2}\alpha b^\dagger b(b^\dagger b - 1) + g(cb^\dagger + c^\dagger b) + a_z(t)b^\dagger b + a_x(t)(b + b^\dagger). \tag{54}$$

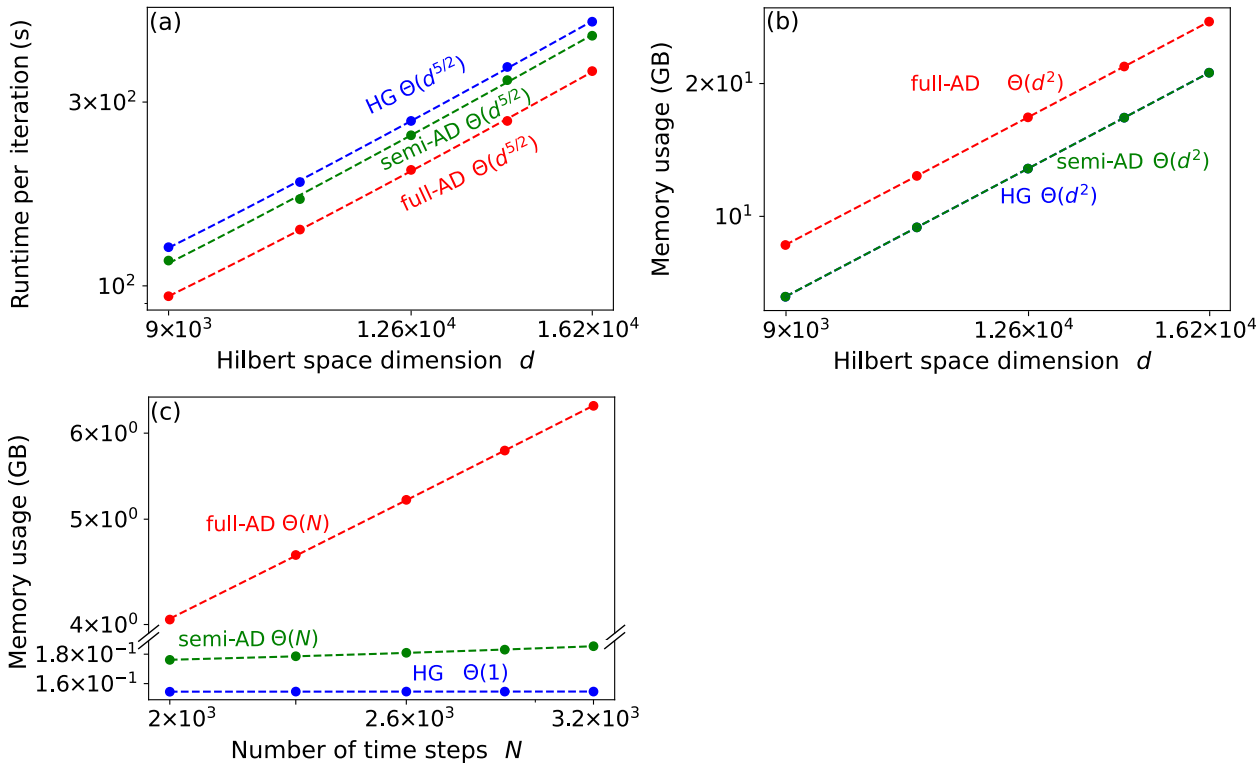

Figure 3: Benchmark results for Fock state generation in a system consisting of a cavity coupled to a transmon. (a) Runtime per iteration and (b) memory usage versus Hilbert space dimension d. The dimension is varied by increasing cavity dimension while fixing the transmon dimension to 6. (c) Memory usage as a function of the number of time steps for $d = 240$. All plots are in log-log scale. Dashed lines are power-law fits. (Parameters: $\Delta/2\pi = 3\,\text{GHz}$, $g/2\pi = 0.1\,\text{GHz}$, $\alpha/2\pi = -0.225$ GHz and constant controls $a_z/2\pi = a_x/2\pi = 0.1$ GHz.)

Here, the drive is both longitudinal and transverse, c and b are the annihilation operators for cavity photons and transmon excitations respectively. α, g and Δ denote anharmonicity, interaction strength and difference between the transmon and cavity frequency.

Optimization proceeds by minimizing the infidelity C_1^{st} and limiting the occupation of higher transmon levels by including the cost contribution C_2^{st} [Eq. (11)]. The latter has the additional benefit of enabling the truncation to only a few transmon levels. We measure runtime and peak memory usage of full-AD[5], semi-AD and HG for a single optimization iteration. The runtime is measured by the package `Temci` [45] and the peak memory usage is monitored by the `memory_profiler` package [48]. The optimization is performed on an AMD Ryzen Threadripper 3970X 32-Core CPU with 3.7 GHz frequency.

Benchmark results for the state-transfer optimization are shown in Fig. 3. We consider a single time step ($N = 1$) and measure the runtime versus d. The results in Fig. 3(a) confirm that full-AD, semi-AD and HG have the same scaling of runtime per time step with respect to d (Tab. 1). The observed runtime scaling $\Theta(d^{\frac{5}{2}})$ is consistent with $\kappa \sim \Theta(d^2)$ due to dense matrix storage for H_1, and $\mu \sim \Theta(d^{\frac{1}{2}})$ (see A). The scaling of the required memory resources is illustrated in Fig. 3(b): memory usage scales with dimension d as $\Theta(d^2)$ for full-AD, semi-AD and HG, consistent with $\kappa \sim \Theta(d^2)$ being the dominant contribution. As anticipated, the scaling of memory with N differs between full-AD, semi-AD and HG [Fig. 3(c)]. For full-AD and semi-AD, the scaling is linear with N, while it is independent of N for HG. The slope of full-AD is much larger than the one of semi-AD, since the former has extra dependence on μ (Tab. 1), which is a constant ≈ 100 in this benchmark. The results confirm the favorable memory efficiency of HG and semi-AD compared to full-AD in state-transfer problems with large number of time steps.

We next turn to an example for optimizing a unitary gate. This benchmark is performed for three driven transmons with nearest neighbor coupling. The target gate consists of simultaneous Hadamard operations on each of the three transmons. For concreteness, we consider the case of transmons with the same frequency, driven resonantly. The full Hamiltonian in the rotating frame

[5] AD is implemented by the package `Autograd` [39] in this benchmark. This package does not support sparse linear algebra, so we implement dense matrix storage for the Hamiltonian in both full-AD and HG for fair comparison.

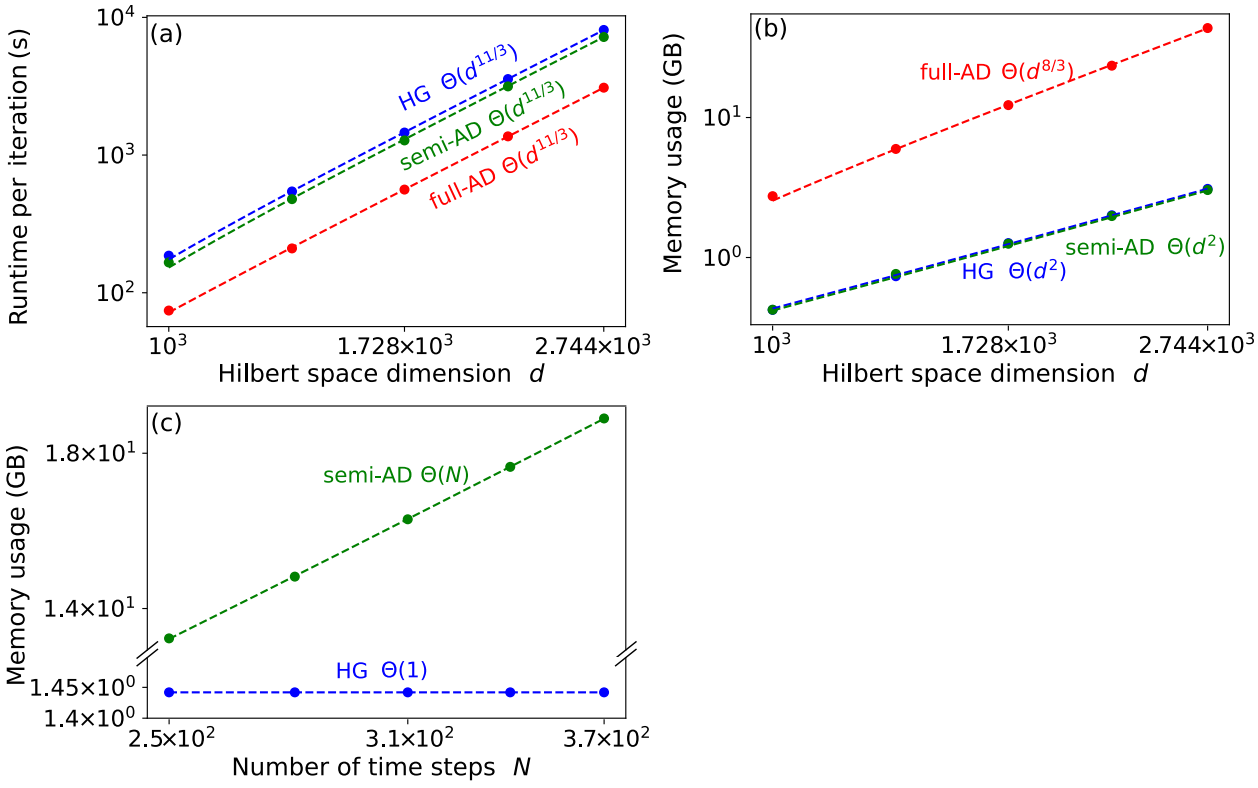

Figure 4: Benchmark results for unitary gate optimization in a system consisting of three coupled transmons. (a) Runtime per iteration and (b) memory usage versus Hilbert space dimension d. The total Hilbert space dimension d is increased by enlarging the cutoff of each transmon from 10 to 14 simultaneously. (c) Memory usage as a function of the number of time steps for $d = 12^3 = 1728$. Full-AD (not shown) would require more memory than our resources (estimated: 1TB). All plots are in log-log scale. Dashed lines are power-law fits. (Parameters: $g/2\pi = 0.1$GHz, $\alpha/2\pi = -0.225$ GHz and $a^{(\nu)}/2\pi = 0.1$ GHz.)

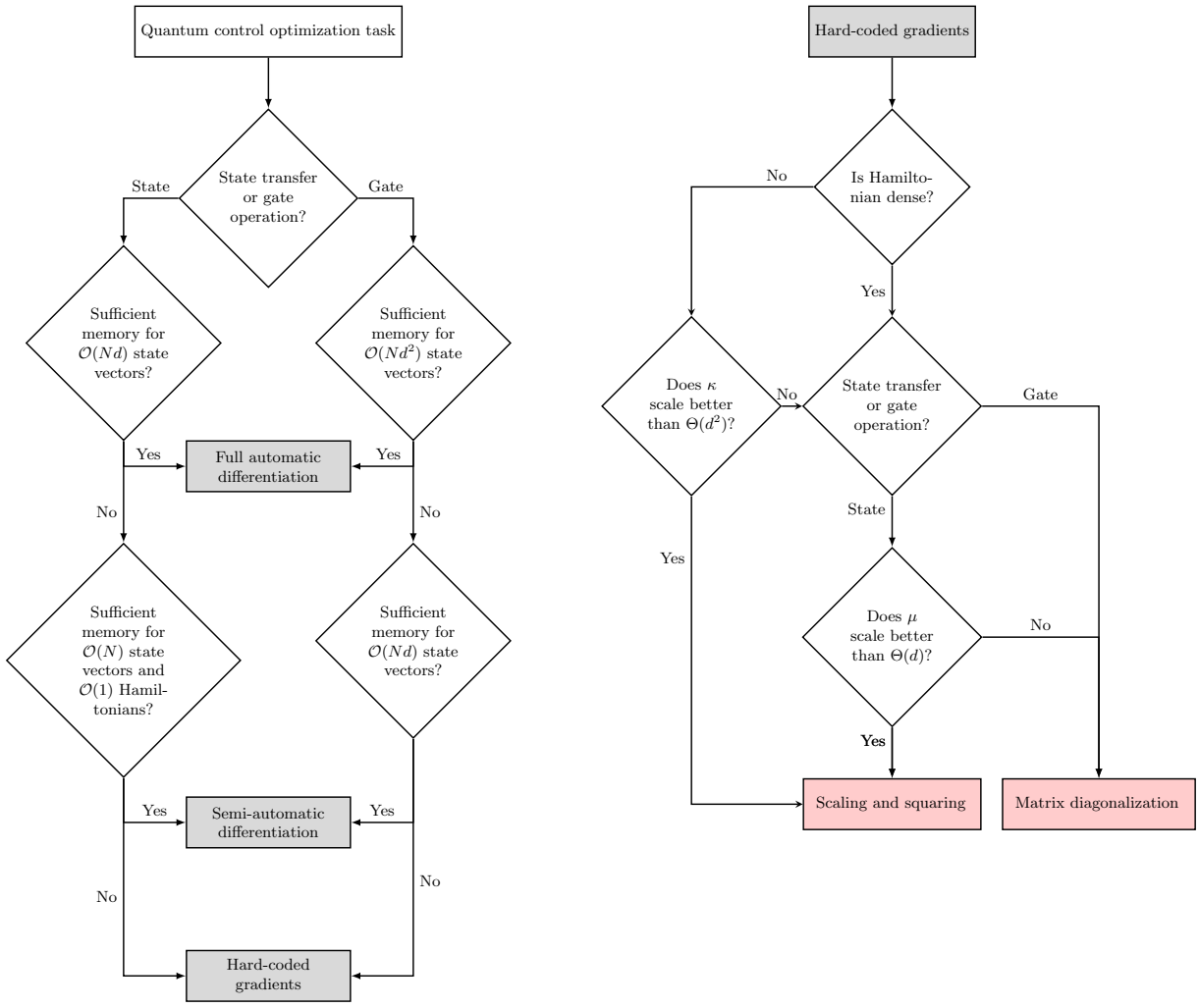

Figure 5: Decision trees for selecting the optimal numerical strategy. (a) Choosing among three numerical methods for evaluating gradients: full automatic differentiation, semi-automatic differentiation, hard-coded analytical gradients. (Here, we we assume $\mu \sim \Theta(d)$; for different scaling see Tab. 1.) (b) Determining the appropriate strategy for evaluating propagator-state products.

of the drive is

$$H_2(t) = \sum_{\nu=1}^{3} \left[\frac{1}{2} \alpha b_\nu^\dagger b_\nu (b_\nu^\dagger b_\nu - 1) + a^{(\nu)}(t)(b_\nu + b_\nu^\dagger) \right] + g(b_1 b_2^\dagger + b_2 b_3^\dagger + \text{h.c.}). \tag{55}$$

The goal of the optimization is to minimize the infidelity C_2^g. The setup for this benchmark is the same as in the previous example.

Fig. 4 records the benchmarks based on a single time step ($N = 1$). The observed runtime scaling with dimension d is $\Theta(d^{\frac{11}{3}})$ for full-AD, semi-AD and HG, see Fig. 4(a). This power law arises from $\Theta(\mu\kappa d)$ for $\kappa \sim \Theta(d^2)$ and $\mu \sim \Theta(d^{\frac{2}{3}})$ [see A]. The memory usage scaling with d is illustrated in Fig. 4(b), confirming that full-AD has an unfavorable extra $\mu \sim \Theta(d^{\frac{2}{3}})$ dependence compared to HG and semi-AD (Tab. 1). The memory usage scaling with N is as expected [Fig. 4(c)]: for semi-AD, the scaling is linear with N; for HG, it is independent of N. (The memory usage of full-AD in this case exceeds our resources, and thus is not shown.) Throughout our benchmark, this linear dependence with N causes semi-AD to consume about 10 times more memory than HG. This underlines that HG is preferable whenever memory constraints become a limitation in optimization problems with large Hilbert space dimension and large number of time steps. We emphasize that this improvement is achieved without worsening the runtime scaling.

4.3 Choosing among different optimization strategies

There are several considerations which determine whether full-AD, HG or semi-AD is the optimal strategy for a given optimization problem, see the decision tree Fig. 5. Whenever the Hilbert space dimension d and the number of time steps N are sufficiently small, then memory usage may not be a bottleneck. In this case, full-AD may be preferable, since it eliminates the need for hard-coded gradients. If the memory cost of full-AD is not affordable but analytical gradients are tedious to compute, semi-AD may be a viable compromise, though still less memory efficient than HG. In all other cases, HG is the method of choice, since it can handle optimization with larger N and d

compared to full-AD and semi-AD.

The choice of implementing HG via scaling and squaring or matrix diagonalization should be informed by whether the Hamiltonian is sparse and κ scales better than $\Theta(d^2)$. In the latter case, scaling and squaring is preferable and leads to an overall memory usage $\sim \Theta(d+\kappa)$, which is better than the memory scaling $O(d^2)$ of matrix diagonalization. By contrast, for Hamiltonians with $\kappa \sim \Theta(d^2)$, these two methods are equivalent in memory usage scaling and we are free to select the one with better runtime for a specific optimization task. While the runtime for optimization using matrix diagonalization scales as $O(d^3)$ [B], the runtime for optimization based on scaling and squaring differs between state-transfer and gate optimizations: (1) for state transfer, the runtime scales as $\Theta(\mu d^2)$; hence, whenever μ scales better than $\Theta(d)$, scaling and squaring wins over matrix diagonalization. (2) for gate optimization, the runtime scales as $\Theta(\mu d^3)$; hence matrix diagonalization is preferable.

5 Application of GRAPE: Robust control for transmon

Transmon has bene widely used as either qubit or ancilla for bosonic qubit. In both cases, X_θ gate in the first two level subspace of transmon is crucial for realizing universal gate in the composite system. One important preliminary to obtain high-fidelity gate in the composite system is to acquire robust X_θ in single transmon system[].

Detuning-robust control for two-level systems has been extensively investigated using various approaches, including reverse engineering [49, 50], space curve quantum control [51], pulse engineering [51–59], and the collocation method [60, 61]. However, protocols developed for two-level systems cannot be directly applied to weakly-anharmonic transmons due to leakage out of the qubit subspace. One promising solution is to apply the DRAG scheme to robust single-quadrature pulses developed for two-level systems [62]. Additionally, Ref. [63] directly optimizes two-quadrature pulses for transmons. In following, we introduce an alternative optimization-based procedure to design detuning-robust control for transmon $\hat{X}_\theta$ gates and benchmark its performance.

5.1 Metrics of robust control

During the gate operation, the Hamiltonian of interest may depend on an unknown quantity δ. This uncertainty can cause the realized gate to deviate from the target gate, thus generally lowering gate fidelity $\mathcal{I}(\delta)$. To quantify the robustness of the infidelity against δ, we consider both the mean infidelity and its standard deviation as key metrics,

$$\mathcal{I}_\mu = \int_{\mathbb{D}} \mathrm{d}\delta\, p(\delta)\mathcal{I}(\delta), \quad \mathcal{I}_\sigma = \sqrt{\int_{\mathbb{D}} \mathrm{d}\delta\, p(\delta)\big[\mathcal{I}(\delta) - I_\mu\big]^2}.$$

Here, the probability distribution $p(\delta)$ satisfies $\int_{\mathbb{D}} \mathrm{d}\delta\, p(\delta) = 1$ for a specified range $\mathbb{D}$.

In our case of interest, the Hamiltonian governing the transmon qubit in the co-rotating frame of the drive frequency is

$$\hat{H}_{\text{trans}}(t) = K_{\text{q}}\hat{q}^{\dagger 2}\hat{q}^2/2 + \varepsilon_I(t)(\hat{q}^\dagger + \hat{q}) + i\varepsilon_Q(t)(\hat{q}^\dagger - \hat{q}) + \delta\hat{q}^\dagger\hat{q}.$$

Here, $\varepsilon_I(t)$ and $\varepsilon_Q(t)$ describe control envelopes in the I and Q quadratures, and δ denotes the difference between the drive frequency and unknown shifted qubit frequency. The closed-system gate infidelity follows

$$\mathcal{I}_{\text{c}}(\delta) = 1 - \frac{1}{4}\left|\mathrm{Tr}\left[\hat{P}_{0,1}\hat{U}^\dagger_{\text{target}}\hat{U}_\delta(t_{\text{g}})\right]\right|^2, \tag{56}$$

where $\hat{P}_{0,1}$ is a projector onto the computational subspace formed by the lowest two eigenstates. We define $\hat{U}_{\text{target}}$ as the target unitary on the transmon, and $\hat{U}_\delta(t_{\text{g}}) = \mathcal{T}\exp\Big[-i\int_0^{t_{\text{g}}}\mathrm{d}\tau\,\hat{H}_{\text{trans}}(\tau)\Big]$ as the actual unitary achieved over a gate duration t_{g}, where $\mathcal{T}$ is the time-ordering operator.

In the presence of decoherence, the open-system infidelity follows [64]

$$\mathcal{I}_{\text{o}}(\delta) = 1 - \frac{1}{4}\mathrm{Tr}\Big[\mathcal{P}_{0,1}\mathcal{L}^\dagger_{\text{target}}\mathcal{L}_\delta(t_{\text{g}})\Big]. \tag{57}$$

Here, the density matrix is vectorized by stacking the elements row-by-row. $\mathcal{P}_{0,1} = \hat{P}_{0,1}\otimes\hat{P}_{0,1}$ is the superprojector onto the computational subspace, $\mathcal{L}_{\text{target}} = \hat{U}_{\text{target}}\otimes\hat{U}^*_{\text{target}}$ is the target superoperator,

and $\mathcal{L}_\delta(t_\mathrm{g})$ is the superoperator realized by the Lindblad master equation. Note that in the absence of decoherence, we have $\mathcal{L}_\delta(t_\mathrm{g}) = \hat{U}_\delta(t_\mathrm{g}) \otimes \hat{U}_\delta^*(t_\mathrm{g})$.

5.2 Optimization procedure

Quantum optimal control has been extensively used to develop robust control schemes by minimizing cost functions that encode robustness metrics. One approach is to penalize the derivative of the closed- or open-system fidelity with respect to detuning δ [62, 63, 65]. However, a more intuitive and pragmatic method is to penalize the average infidelity [66–70]

$$C_{\mathrm{o,c}} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{I}_{\mathrm{o,c}}(\delta_i). \tag{58}$$

Here, δ_i denotes sample values of detunings within the specified range of $\mathbb{D}$. We find that minimization of the above cost function can suppress both mean infidelity and its standard deviation, as evidenced by subsequent numerical results.

To minimize the cost function, we optimize the pulses, including both the envelopes $\varepsilon_{I,Q}(t)$ and the pulse duration t_g. An optimal duration is determined by balancing two competing factors: a shorter duration leads to unwanted leakage, whereas a longer duration results in decoherence errors. A direct approach to optimize pulses involves using an open-system optimizer for various durations, and selecting the one that minimizes the cost function C_o. However, such optimization requires time-intensive open-system simulations. Instead, we propose a method, while not guaranteed to be equivalent, yields promising results. The method consists of two steps: (1) employing closed-system optimization to minimize C_c for each potential duration, and (2) with given decoherence rates, evaluating C_o for the optimized pulses from first step, then selecting the pulse with the lowest C_o. Our approach is summarized in Fig. 6.

For the closed-system optimization, we use Gradient Ascent Pulse Engineering (GRAPE), a method widely employed for optimal control in various contexts [69, 71, 72]. We calculate the gradients required in GRAPE by using automatic differentiation, thereby eliminating the need for

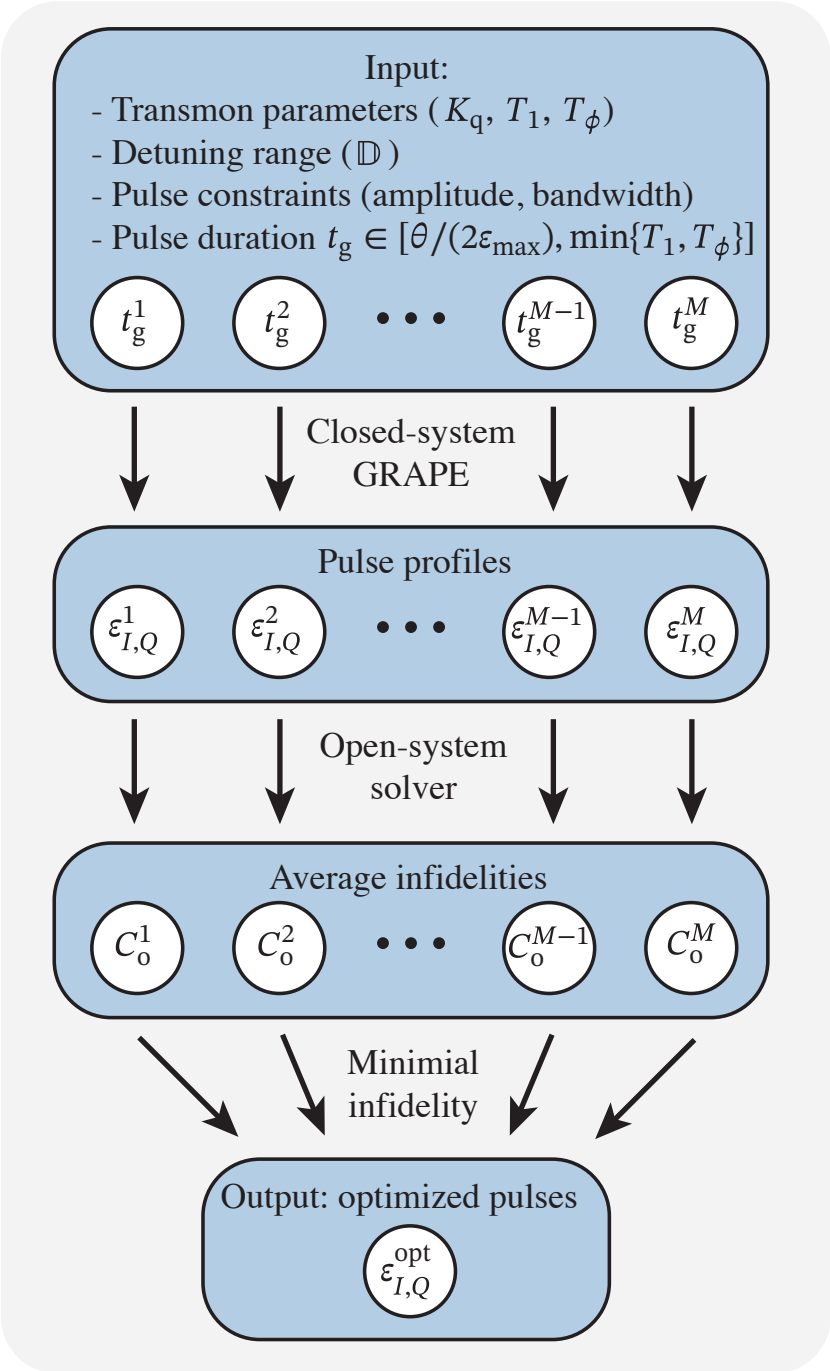

Figure 6: Optimization procedure to obtain detuning-robust $\hat{X}_\theta$ gates in transmons. Input includes pulse constraints, transmon parameters, and a set of M pulse durations. The choice of duration should be shorter than decay time T_1 and longer than $1/\epsilon_{max}$, where ϵ_{max} is the maximum pulse amplitude. Closed-system GRAPE is performed for each duration to obtain the corresponding pulse profiles. Subsequently, an open-system solver evaluates the cost function C_o for pulses with varying durations, selecting the pulse that minimizes C_o.

manual coding the analytical gradients of the cost function [73–75]. We use the initial pulse ansatz

$$\begin{aligned}\varepsilon_I^{\text{initial}}(t) = \frac{1}{t_{\text{g}}}\big[&(a-\theta/2)\cos(2\pi t/t_{\text{g}}) + (b-a)\cos(4\pi t/t_{\text{g}}) \\ &+ (c-b)\cos(6\pi t/t_{\text{g}}) - c\cos(8\pi t/t_{\text{g}}) + \theta/2\big], \\ \varepsilon_Q^{\text{initial}}(t) = -\dot{\varepsilon}_I^{\text{initial}}(t)/K_{\text{q}},&\end{aligned}$$

where θ is the rotation angle for the target $\hat{X}_\theta$ gate ($\theta = \pi, \pi/2$ in our study). This ansatz for I-quadrature control has been used to obtain robust control for two-level systems [59]. The initial guess for the Q-quadrature is inspired by the DRAG pulse scheme [76] to suppress leakage. Then, we optimize the pulse starting from various initial guesses for parameters $a, b, c \in \{-10, -8, -6, \cdots, 10\}$ and select the pulse with the lowest cost value C_{c}. We accelerate this process by parallelizing with different initial values.

Throughout the closed-system optimization process, we ensure pulse smoothness by filtering out frequency components that exceed maximum allowed bandwidths of $5/t_{\text{g}}$ and $10/t_{\text{g}}$ for $\varepsilon_I(t)$ and $\varepsilon_Q(t)$, respectively. These empirical values effectively balance the pulse smoothness and the level of robustness required for our study.

5.3 Benchmarking and comparison

We benchmark our optimization procedure for a transmon with typical parameters: $K_{\text{q}}/2\pi = -200$ MHz, $T_1 = 50$ μs, and $T_\phi = 50$ μs. The optimization results for the $\hat{X}_\pi$ gate are shown in Fig. 7. Figure 7(a) shows the average infidelities obtained for closed- and open-system simulations with varying pulse durations. Without decoherence, extending the pulse duration generally leads to a reduction in the cost value. In contrast, in the presence of decoherence, utilizing shorter pulses is more effective in reducing the infidelity. Given the particular transmon parameters under consideration, a pulse duration of 20 ns achieves the lowest cost. The temporal profiles and frequency components of the optimized 20-ns pulse are shown in Fig. 7(b) and (c), respectively. In Fig. 7(d), we study the infidelity as a function of detuning, comparing the performance between the optimized

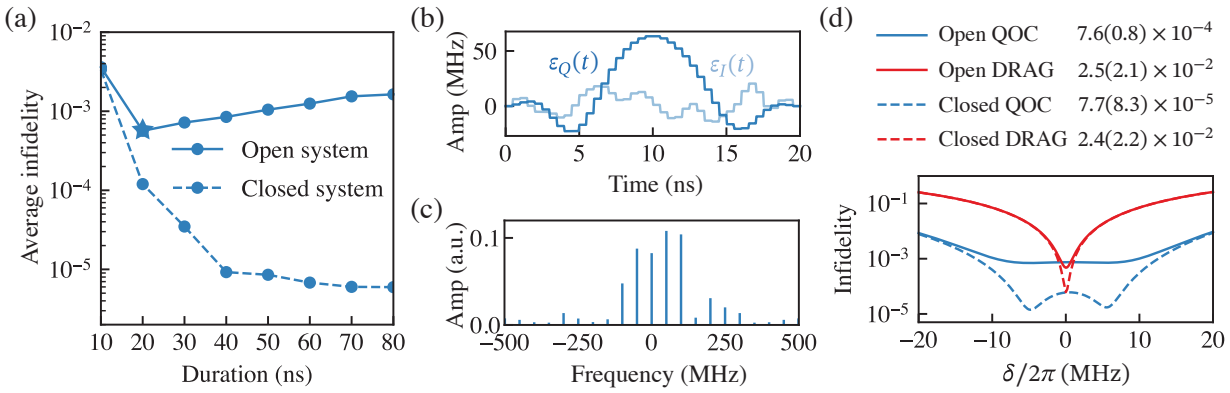

Figure 7: Optimization results for the $\hat{X}_\pi$ gate. (a) Average infidelities in Eq. (58) versus pulse durations for both closed (dashed line) and open (solid line) systems. The optimal duration selected from the range considered for this optimization is 20 ns (indicated by a star). (b) I and Q quadratures of the pulse envelopes for the selected 20-ns pulse from (a). (c) The frequency components of the pulse envelopes, corresponding to the 20-ns pulse. (d) Comparative analysis of the 20-ns pulses, from both QOC (blue) and DRAG (red) pulses, in terms of their infidelities as a function of detuning. The average and standard deviation of the infidelities are tabulated correspondingly by considering a uniform distribution of $\rho(\delta)$ for simplicity. The result shows that the optimized pulse is more robust than the commonly used DRAG pulse. [Parameters: $\delta_i/2\pi \in \{\pm1, \pm3, \pm4, \pm7, \pm10\}$ MHz, $K_\mathrm{q}/2\pi = -200$ MHz, $T_1 = 50$ μs, $T_\phi = 50$ μs.]

and the DRAG pulses. In both closed- and open-system simulations, the optimized pulse consistently demonstrates substantially reduced infidelities across almost all detunings. Consequently, the optimized pulse exhibits enhanced robustness against frequency detuning in the transmon ancilla. The open-system infidelities obtained from these two pulses are limited by distinct factors: the infidelity derived from the QOC pulse is dominated by decoherence errors, while the one generated from the DRAG pulse is limited by coherent error.

We then compare the robustness of the pulses derived from our optimization procedure with selected recent literature. First, we follow the methodology outlined in Ref. [63] to obtain robust pulses using the `Q-CTRL` package [57, 77]. To distinguish between these results and those generated by our optimization procedure, we use the label "Q-CTRL" for the former and "QOC" for the latter. The averaged infidelities [Eq. (58)] of the closed system as a function of pulse duration are shown in Fig. 8(a). While the average infidelities for Q-CTRL (red) and QOC (blue) are similar for shorter durations, QOC demonstrates orders of magnitude lower infidelity for longer

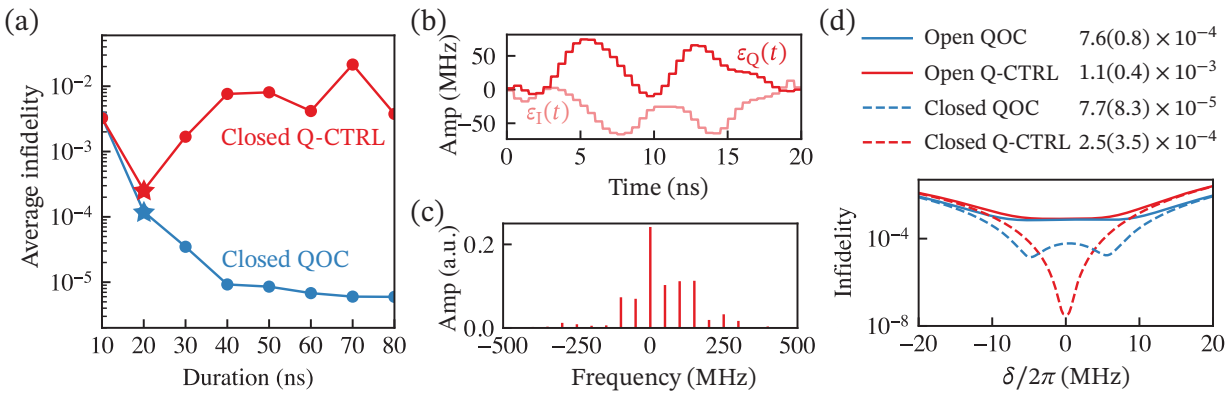

Figure 8: Comparison of optimization results between Q-CTRL and QOC for the $\hat{X}\pi$ gate. (a) Average infidelities of a closed system [Eq. (58)] versus pulse durations derived from Q-CTRL (red) and QOC (blue). The optimal duration selected for Q-CTRL from the range considered is 20 ns (indicated by a red star). (b) I and Q quadratures of the Q-CTRL pulse envelopes for the selected 20-ns pulse from (a). (c) Frequency components of the Q-CTRL pulse envelopes corresponding to the 20-ns pulse. (d) Comparative analysis of the 20-ns pulses from both QOC (blue) and Q-CTRL (red) in terms of their infidelities as a function of detuning. The average and standard deviation of the infidelities are tabulated considering a uniform distribution of $\rho(\delta)$ for simplicity. [Parameters: $\delta_i/2\pi \in \pm1, \pm3, \pm4, \pm7, \pm10$ MHz, $K\mathrm{q}/2\pi = -200$ MHz, $T_1 = 50$ μs, $T_\phi = 50$ μs.]

durations. To illustrate the robustness details of the Q-CTRL results, we examine the 20-ns case, which exhibits the lowest infidelity. The I and Q quadratures of the pulse envelopes and their corresponding Fourier components are shown in Fig. 8(b) and (c). The infidelity as a function of detuning is presented in Fig. 8(d). For a closed system, the Q-CTRL results (red dashed line) show lower infidelity near resonance but exhibit higher infidelity off-resonance ($\delta/2\pi > 5$ MHz) compared to the QOC results (blue dashed line). This difference likely arises from the different cost functions used in the two optimizations: Q-CTRL penalizes the derivative of the infidelity at zero detuning, whereas QOC minimizes the average infidelity over a sampling of detunings ($\delta_i/2\pi \in \{\pm1, \pm3, \pm4, \pm7, \pm10\}$ MHz in the example). Notably, in the presence of ancilla decoherence ($T_1 = 50$ μs, $T_\phi = 50$ μs), the infidelity at small detuning is dominated by incoherent error. This makes QOC the overall better choice within the considered range of detuning for robust pulses.

A The scaling of number of matrix-vector multiplications μ

The asymptotic behavior of runtime, discussed in the main text, is governed by the scaling of μ (50) with dimension d of the Hilbert space. The dependence $\mu(d)$ is specific to the matrix A to be exponentiated (which is proportional to the Hamiltonian). This prevents us from stating a general scaling relation. We thus turn to a discussion of several concrete examples.

We first consider the example of the transmon-cavity system, see Eq. (54), where we increase d via the cavity dimension d_c while leaving the transmon cutoff fixed. The corresponding Hamiltonian H_1, written in the bare product basis, then has a constant maximum number of non-zero elements in each row. (I.e., the parameter σ' (41) is independent of d.) Numerically, we find in this case that μ scales linearly with $||A||_1$ $\big(\mu \sim \Theta(||A||_1)\big)$, and this linearity holds for a wide range of τ, see Fig. 9(a). In this example, two ingredients lead to the scaling $||A||_1 \sim \Theta(d^{\frac{1}{2}})$: (1) d is a constant multiple of d_c, and (2) the cavity ladder operators in H_1 which has one-norm scaling $\Theta(d_c^{\frac{1}{2}})$. Thus, we obtain the overall scaling $\mu \sim \Theta(d^{\frac{1}{2}})$, see Fig. 9(e). In the second example, we consider the system three coupled transmons, see Eq. (55). The Hamiltonian H_2 is written in the harmonic oscillator basis and has a constant σ', which leads to $\mu \sim \Theta(||A||_1)$ [Fig. 9(b)]. One-norm of operator $(b^\dagger b)^2$ in H_2 has quadratic scaling with each transmon dimension. Since we increase d by enlarging the dimensions of all three transmons simultaneously, we obtain $||A||_1 \sim \Theta(d^{\frac{2}{3}})$. This scaling results in $\mu \sim \Theta(d^{\frac{2}{3}})$, see Fig. 9 (f).

For other Hamiltonian matrices where σ' depends on d, the scaling $\mu \sim \Theta(||A||_1)$ may still hold. In the following, we show two concrete examples that one where linearity holds, and another where it breaks. We first consider a driven system of N_q qubits with nearest-neighbor σ_z coupling. In the rotating frame, the system is described by

$$H_3(t) = \sum_{\nu=1}^{N_q} [a_x^{(\nu)}(t)\sigma_x^{(\nu)} + a_y^{(\nu)}(t)\sigma_y^{(\nu)}] + \sum_{\nu=1}^{N_q-1} g\sigma_z^{(\nu)}\sigma_z^{(\nu+1)}.$$

Each qubit is controlled via drive fields $a_x^{(n)}$ and $a_y^{(n)}$. For this example, the numerical results show that scaling $\mu \sim \Theta(||A||_1)$ still holds, see Fig. 9(c). Since we increase d by enlarging N_q, we

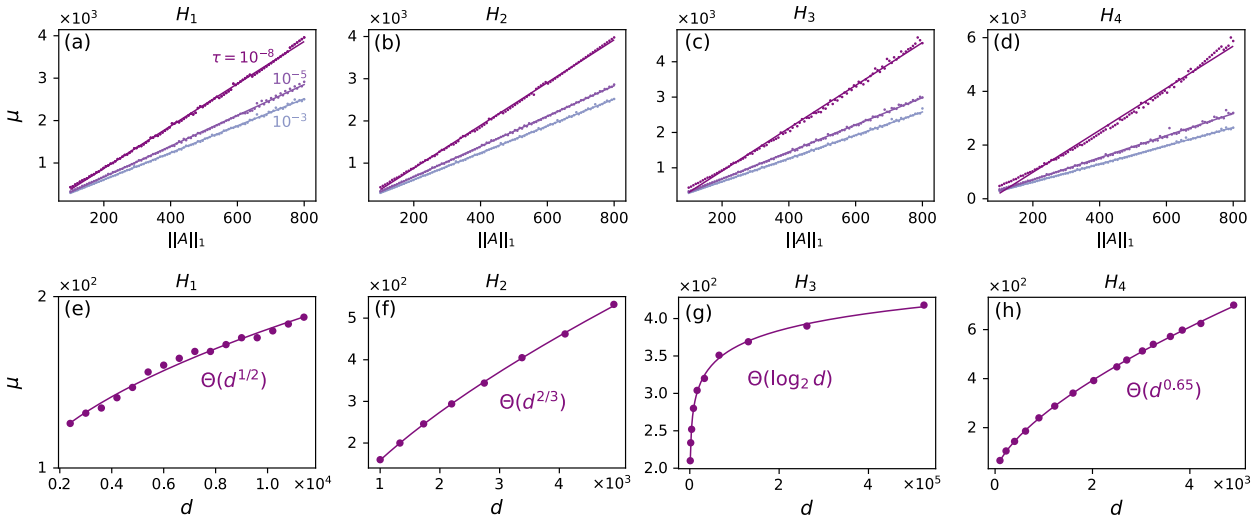

Figure 9: Scaling of μ, the number of matrix-vector multiplications. (a)-(d) The scaling of μ with $||A||_1$ for Hamiltonians H_j, $j = 1, 2, 3, 4$, when considering several error tolerances τ. (e)-(h) The scaling of μ with dimension d when considering $\tau = 10^{-8}$ for each H_j, respectively. In all figures, dots are sampling data and solid lines are fitting curves. The parameters setup of H_1, H_2 are the same as the ones given in the captions of Figs. 3, 4. Parameters of H_3 and H_4 are as follows: for H_3, $E_{Ci}/2\pi = 2.5$ GHz, $E_{Ji}/2\pi = 8.9$ GHz, $E_{Li}/2\pi = 0.5$ GHz, $g/2\pi = 0.1$ GHz and $\phi = 0.33$; for H_4, $a_x^{(\nu)}/2\pi = a_y^{(\nu)}/2\pi = 0.5$ GHz and $g/2\pi = 0.1$ GHz.

obtain $||A||_1 \sim \Theta(N_q) \sim \Theta(\log_2 d)$. This scaling results in $\mu \sim \Theta(\log_2 d)$, see Fig. 9(g). For the second example, we consider two capacitively coupled fluxonium qubits with Hamiltonian

$$H_4(t) = \sum_{i=1}^{2} \left(4E_{Ci}n_i^2 - E_{Ji}\cos(\varphi_i) + \frac{E_{Li}}{2}[\varphi_i - \phi_i(t)]^2\right) + gn_1n_2. \tag{59}$$

Here, E_{Ci}, E_{Ji}, E_{Li} denote the charging, inductive and Josephson energies of two fluxonium qubits, and g is the interaction strength. φ and n are the phase and charge number operators, defined in the usual way. We represent the Hamiltonian in the harmonic oscillator basis. Fig. 9(d) shows that scaling of μ is superlinear with $||A||_1$ for $\tau = 10^{-8}$. We thus settle for a numerical determination of the scaling relation between μ and d for $\tau = 10^{-8}$ by increasing the dimensional cutoff for both fluxonium qubits simultaneously. The observed scaling is $\mu \sim \Theta(d^{0.65})$, see Fig. 9(h).

B Alternative HG Scheme: Numerical Implementation and Scaling

The numerical implementation of the hard-coded gradient scheme requires evaluation of the short-time propagator and its derivative. Under certain conditions, it turns out to be advantageous to numerically evaluate the short-time propagator by employing matrix diagonalization. In this appendix, we first discuss the method to compute propagator derivative and then analyze the scaling of HG.

B.1 Evaluation of propagator derivative

The operator $-iH\Delta t$ is anti-Hermitian, and hence can be diagonalized and exponentiated according to

$$A = SDS^\dagger, \tag{60}$$

$$e^A = Se^D S^\dagger. \tag{61}$$

Here, S is unitary and D diagonal. As a result, the propagator derivative can be written as

$$\frac{de^A}{da} = Se^D \frac{dD}{da} S^\dagger + \frac{dS}{da} e^D S^\dagger + Se^D \frac{dS^\dagger}{da}. \tag{62}$$

We apply the inverse unitary transformation to $\frac{de^A}{da}$ (62) and obtain

$$S^\dagger \frac{de^A}{da} S = e^D \frac{dD}{da} + S^\dagger \frac{dS}{da} e^D + e^D \frac{dS^\dagger}{da} S. \tag{63}$$

Based on $\frac{dS^\dagger}{da} S + S^\dagger \frac{dS}{da} = 0$, Eq. (63) can be rewritten as

$$S^\dagger \frac{de^A}{da} S = e^D \frac{dD}{da} + E \circ (S^\dagger \frac{dS}{da}) \tag{64}$$

with $E_{ij} := e^{D_{jj}} - e^{D_{ii}}$. Here, $\circ$ denotes the Hadamard product (element-wise multiplication). To obtain dD/da and dS/da, we take the derivative of Eq. (60) and repeat the same steps as for Eqs. (63) and (64), leading to

$$S^\dagger \frac{dA}{da} S = \frac{dD}{da} + E' \circ (S^\dagger \frac{dS}{da}), \tag{65}$$

where $E'_{ij} := D_{jj} - D_{ii}$. Since the diagonal elements of E' are zero, it follows that

$$\frac{dD}{da} = I \circ (S^\dagger \frac{dA}{da} S). \tag{66}$$

For off-diagonal elements of the operator in Eq. (65), we obtain

$$F \circ (S^\dagger \frac{dA}{da} S) + G = S^\dagger \frac{dS}{da}, \tag{67}$$

where we define

$$F_{ij} := \begin{cases} \frac{1}{D_{jj} - D_{ii}}, & \text{if } D_{jj} - D_{ii} \neq 0. \\ 0, & \text{otherwise.} \end{cases} \tag{68}$$

$$G_{ij} := \begin{cases} 0, & \text{if } D_{jj} - D_{ii} \neq 0. \\ (S^\dagger \frac{dS}{da})_{ij}, & \text{otherwise.} \end{cases}$$

Inserting Eqs. (66) and (67) into Eq. (64) gives

$$\frac{de^A}{da} = S\big((e^D + E \circ F) \circ (S^\dagger \frac{dA}{da} S)\big) S^\dagger, \tag{69}$$

where we use $E \circ G = 0$.

B.2 Scaling analysis

The gradients of the contributions to the cost function (for the case of state transfer) are evaluated by the forward-backward propagation scheme. In this scheme, the storage required for states does not scale with the number of intermediate time steps. The matrices that must be stored are the system and control Hamiltonians as well as dense matrices dense matrices S, E and E'. Thus, the total memory usage scales as $O(d^2)$. The runtime of the scheme is dominated by $O(N)$ evaluations of the propagator and its derivative. Employing matrix diagonalization for the propagator evaluation requires a runtime scaling $\sim O(d^3)$ [78]. Calculation of the propagator derivative involves matrix-matrix multiplication and Hadamard product with runtime scaling $O(d^3)$ and $O(d^2)$, respectively. Thus, the overall runtime of the HG scheme scales as $O(Nd^3)$.

The gradients of the contributions to the gate-operation cost function are evaluated in an analogous manner, resulting in the runtime and memory scaling.

References

[1] Rojan et al., Physical Review A **90**, 023824 (2014).

[2] D. I. Bondar et al., Globally optimal quantum control (2023), arXiv:2209.05790 [quant-ph] .

[3] Waldherr et al., Nature **506**, 204 (2014).

[4] Gertler et al., Nature **590**, 243 (2021).

[5] Abdelhafez et al., Physical Review A **101**, 022321 (2020).

[6] Allen et al., Physical Review A **95**, 042325 (2017).

[7] Huang et al., Physical Review A **90**, 012318 (2014).

[8] Werninghaus et al., npj Quantum Information **7**, 14 (2021).

[9] Glaser et al., The European Physical Journal D **69**, 279 (2015).

[10] Bretschneider et al., The Journal of Chemical Physics **136**, 094201 (2012).

[11] Kehlet et al., Journal of the American Chemical Society **126**, 10202 (2004).

[12] Khaneja et al., Journal of Magnetic Resonance **172**, 296 (2005).

[13] Tošner et al., Journal of Magnetic Resonance **197**, 120 (2009).

[14] Hogben et al., Journal of Magnetic Resonance **208**, 179 (2011).

[15] Xu et al., Magnetic Resonance in Medicine **59**, 547 (2008).

[16] Müller et al., Physical Review A **92**, 053423 (2015).

[17] Nebendahl et al., Physical Review A **79**, 012312 (2009).

[18] A. Angerer, Robust coherent optimal control of nitrogen-vacancy quantum bits in diamond, Ph.D. thesis, Vienna University of Technology (2013).

[19] Chou et al., Physical Review A **91**, 052315 (2015).

[20] Dolde et al., Nature Communications **5**, 3371 (2014).

[21] Poggiali et al., Physical Review X **8**, 021059 (2018).

[22] Poulsen et al., Physical Review B **106**, 014202 (2022).

[23] Rembold et al., AVS Quantum Science **2**, 024701 (2020).

[24] Yan et al., JETP Letters **114**, 314 (2021).

[25] Amri et al., Scientific Reports **9**, 5346 (2019).

[26] Mennemann et al., New Journal of Physics **17**, 113027 (2015).

[27] Sørensen et al., Computer Physics Communications **243**, 135 (2019).

[28] Guo et al., Nature Communications **10**, 148 (2019).

[29] Rosi et al., Physical Review A **88**, 021601(R) (2013).

[30] Baydin et al., Journal of Machine Learning Research **18**, 1 (2018).

[31] Leung et al., Physical Review A **95**, 042318 (2017).

[32] T. Propson, Qoc, `https://github.com/SchusterLab/qoc.git` (2019).

[33] Abdelhafez et al., Physical Review A **99**, 052327 (2019).

[34] Zhang et al., Phys. Rev. Res. **4**, 043057 (2022).

[35] Goerz et al., Quantum **6**, 871 (2022).

[36] Arute et al., Nature **574**, 505 (2019).

[37] Gong et al., Science **372**, 948 (2021).

[38] M. Abadi et al., TensorFlow: Large-scale machine learning on heterogeneous systems (2015), software available from tensorflow.org.

[39] D. Maclaurin et al., in ICML 2015 AutoML workshop, Vol. 238 (2015).

[40] Moler et al., SIAM Review **45**, 3 (2003).

[41] Codenotti et al., Calcolo **29**, 1 (1992).

[42] N. J. Higham, Accuracy and stability of numerical algorithms, 4th ed. (SIAM, 2002) Chap. 2–3.

[43] J. H. Wilkinson, Rounding errors in algebraic processes, (Courier Corporation, 1994) p. 82.

[44] Al-Mohy et al., SIAM Journal on Scientific Computing **33**, 488 (2011).

[45] J. Bechberger, Temci, `https://github.com/parttimenerd/temci.git` (2015).

[46] Goodwin et al., The Journal of Chemical Physics **143**, 084113 (2015).

[47] Najfeld et al., Advances in Applied Mathematics **16**, 321 (1995).

[48] F. Pedregosa, Memory profiler, `https://github.com/pythonprofilers/memory_profiler.git` (2012).

[49] Daems et al., Phys. Rev. Lett. **111**, 050404 (2013).

[50] Dridi et al., Phys. Rev. Lett. **125**, 250403 (2020).

[51] Barnes et al., Quantum Sci. Technol. **7**, 023001 (2022).

[52] Wang et al., Nat. Commun. **3**, 997 (2012).

[53] Torosov et al., Phys. Rev. A **83**, 053420 (2011).

[54] Owrutsky et al., Phys. Rev. A **86**, 022315 (2012).

[55] Tycko et al., Phys. Rev. Lett. **51**, 775 (1983).

[56] Wu et al., Phys. Rev. A **107**, 023103 (2023).

[57] Ball et al., Quantum Sci. Technol. **6**, 044011 (2021).

[58] Wimperis et al., J. Magn. Reson. A **109**, 221 (1994).

[59] Pasini et al., Phys. Rev. A **80**, 022328 (2009).

[60] Propson et al., Phys. Rev. Appl. **17**, 014036 (2022).

[61] Trowbridge et al., arXiv:2305.03261 (2023).

[62] Hai et al., arXiv:2210.14521 (2023).

[63] Carvalho et al., Phys. Rev. Appl. **15**, 064054 (2021).

[64] M. R. Abdelhafez, Quantum Optimal Control Using Automatic Differentiation, Ph.D. thesis, The University of Chicago (2019).

[65] Watanabe et al., Phys. Rev. A **109**, 012616 (2024).

[66] P. Reinhold, Controlling error-correctable bosonic qubits, Ph.D. thesis, Yale Univ. (2019).

[67] J. Allen, Robust Optimal Control of the Cross-Resonance Gate in Superconducting Qubits, Ph.D. thesis, Univ. of Surrey (2020).

[68] Kosut et al., Phys. Rev. A **88**, 052326 (2013).

[69] Khaneja et al., J. Magn. Reson. **172**, 296 (2005).

[70] Rembold et al., AVS Quantum Sci. **2**, 024701 (2020).

[71] Lu et al., J. Phys. Commun. **8**, 025002 (2024).

[72] Koch et al., EPJ Quantum Technol. **9**, 19 (2022).

[73] Baydin et al., arXiv:1404.7456 (2014).

[74] Leung et al., Phys. Rev. A **95**, 042318 (2017).

[75] Abdelhafez et al., Phys. Rev. A **99**, 052327 (2019).

[76] Motzoi et al., Phys. Rev. Lett. **103**, 110501 (2009).

[77] Q-CTRL, Boulder opal (2022).

[78] V. Y. Pan et al., in Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, STOC '99 (Association for Computing Machinery, 1999) p. 507–516.